10/26/2026
LRG

Lab 10 - I^2C Communication

•Purpose: Use the PIC16F883 MSSP peripheral for I^2C communication. Begin with an address only master transaction and measurements of the two-wire bus. Later parts add an external sensor, a second PIC operating as a slave, and UART integration.

•Equipment and materials: Part 1- One PIC16F883 minimum circuit with a 4 MHz crystal, 5v supply, PICkit 3 (or equivalent), MPLAB X/pic-as, single I^2C pull up resistors in common values near the four calculated test- targets (two of each chosen value, one per bus line), beardboard components, multimeter, and an oscilloscope with I^2C decoding. Provide scope access to RC3/SCL, RC4/SDA, RA0, and RA1. Later parts- A second PIC16F883 circuit, supplied I^2C sensor breakout and input source, Lab 8 UART host application and serial adapter, ADC sensor and servo components, and optional Mastery hardware.

•Part 1 - I^2C Master and Bus Timing:

•Goal: Use one PIC16F883 as a 100kHz I^2C master. Send a periodic address-only write transaction with no slave connected. Detect the expected NACK, generate STOP, and measure the bus waveforms.

•Required behavior:

• Use a 4 MHz crystal and a 5v PIC16F883 circuit. Connect RC3/SCL (PIN 14) and RC4/SDA (pin 15) to 5v through seperate 4.7k Ω pull-up resistors. Leave the I^2C bus without a slave for this part. Configure the pins for MSSP open-drain operation.

• Configure the MSSP for 7-bit I^2C master mode at 100kHz. Address 0x20 with R/W=0. The address and R/W bit form the transmitted byte 0x40 (0100 0000 binary).

• For every probe, generate START, send the address/R-W byte, read the ACK/NACK result, and generate STOP. With no safe I^2C slave the ninth clock pulse must show a NACK (SDA High).

• Use a hardware timer to start repeated address transmissions. Choose a timer interval that leaves a clear idle gap between the STOP of on transaction and the START of the next. Calculate the expected interval from your chosen timer settings.

• Use RA0 (Pin 2) as a NACK indicator. Clear RA0 at the beginning of each probe; make it HIGH if the slave doesn't acknowledge. Leave it LOW for ACK. Complete the transaction with STOP in either case.

• Use RA1 (Pin 3) as a timer diagnostic. Set RA1 HIGH when the timer event starts a new transaction, just before calling AddressProbe. Set it LOW after AddressProbe returns following STOP. Each RA1 rising edge marks the start of a new transaction and provides an oscilloscope trigger without I^2C decoding.

•Before Lab:

• A schematic with the PIC power, reset, oscillator, programming connections, separate SDA/SCL pull-ups, and RA0/RA1 diagnostic connections. Account for every used pin and component as required by the Lab Standard.

• PIC16F883 SFR documentation for MSSP master setup, START/STOP control, address transmission, complete and ACK status, the selected timer's setup and event indication, and the I/O configuration used here. Include the relevant register maps and the values you plan to use.

• Flow charts for the main loop and AddressProbe subroutine. Show the timer event, RA1 diagnostic signal, START, address transmission, NACK decision, STOP, and return.

• Predicted SCL clock period and frequency and the interval between transactions based on the selected timer.

10/26/2026
LRG

Lab 10 - I²C

• Pull-up resistor investigation: Starting with the nominal 4.7kΩ resistor, calculate the target resistance for nominal, nominal ×10, nominal ÷2, and nominal ÷10. For each target, select and document a nearby common resistor value that is actually available. Use on resistor rating, and predicted electrical results for each test.

• For both SDA and SCL at every selected resistance, predicted the idle High voltage, current through the pull up with the bus idle and with the line driven low, and 30% - 70% rise time. State the assumed bus capacitance and LOW voltage. Compare predictions with the I²C Standard-mode electrical requirements and PIC16F883 pin limits before applying power.

• Compile the Part 1 source and program the PIC before starting the In the Lab steps.

- MSSP timing example: For a 4MHz oscillator and 100KHz I²C clock, find the master baud-rate value using the data sheet equation.

* $f_{SCL} = \frac{F_{osc}}{4(SSPADD+1)}$

* $SSPADD = \frac{F_{osc}}{4 f_{SCL}} - 1 \rightarrow \frac{4MHz}{4(100KHz)} - 1 \rightarrow 10 - 1 = \underline{9}$

* $T_{SCL} = \frac{1}{100KHz} = \underline{10\mu s}$

- Pull up resistor voltage & current example: With neither SDA nor SCL driven low, each line should be close to the 5v supply and the ideal pull up current is zero. For a LOW line, estimate the current through one nominal 4.7kΩ resistor using an assumed 0.4v LOW level.

* $V_{idle} \approx V_{DD} = 5.0v$
* $I_{idle} \approx \frac{V_{DD} - V_{idle}}{R_p}$

$\downarrow$

$\frac{5v - 5v}{4.7K\Omega} \rightarrow 0mA$

* $I_{Low} = \frac{V_{DD} - V_{OL}}{R_p} \rightarrow \frac{5v - 0.4v}{4.7K\Omega} \rightarrow \frac{4.6v}{4.7K\Omega} = \underline{0.979mA}$

- I²C rise time calculation: Begin with the RC charging equation used in class. Let V_i be the initial line voltage, V_f the final voltage, and $\tau = R_p C_{bus}$ the time constant.

* $V(t) = V_f - (V_f - V_i)e^{-t/\tau}$

$\tau = R_p \cdot C_{bus}$

- For the pull up estimate, assume the line starts at $V_i = 0v$ and rises toward $V_f = V_{DD}$. Solve the charging equation for the time at which the line reaches a chosen fraction of V_{DD}:

* $V(t) = V_{DD}(1 - e^{-t/\tau}) \rightarrow \frac{V(t)}{V_{DD}} = 1 - e^{-t/\tau} \Rightarrow e^{-t/\tau} = 1 - \frac{V(t)}{V_{DD}} \rightarrow t = -\tau \ln\left(1 - \frac{V(t)}{V_{DD}}\right)$

0.8473τ = 30% to 70%

- I²C rise time is measured from 30% to 70% of the bus voltage. Calculate the time to reach each threshold, the subtract:

* $t_{30\%} = -\tau \ln(1 - 0.30) \rightarrow -\tau \ln(0.7) \approx 0.356675\tau$
* $t_{70\%} = -\tau \ln(1 - 0.70) \rightarrow -\tau \ln(0.3) \approx 1.203973\tau$
* $t_r = t_{70\%} - t_{30\%} \approx (1.203973 - 0.356675)\tau \approx 0.8473\tau$ or $0.8473 R_p C_{bus}$

- Worked example: Assume a nominal 4.7kΩ pull up and 100pF bus capacitance. The capacitance is an example assumption, not a measured value

* $\tau = R_p C_{bus} = (4.7K\Omega)(100pF) = 470ns \rightarrow t_r \approx 0.8473(470ns) \approx \underline{398.231ns}$

10/26/2026
LRG

Lab 10 - I²C

• Standard-mode I²C specifies a maximum 1µs rise time, and its LOW-level current specification is based on 3mA. The lowest-resistance test can require substantially more LOW-state current. Check whether each chosen resistor meets the PIC pin current/voltage limits and I²C electrical requirements; explain any out of specification prediction. Do not power a circuit that exceeds the PIC's electrical limits. The circuit has wiring, pin, and oscilloscope-probe capacitance even with no slave connected.

• Required setup & Program structure: see Lab assignment in github.

• In the Lab:

1) Build the circuit. Check the supply setting, ground, SDA/SCL pull ups, and connections before applying power, the measure 5v at the PIC. Leave the bus without a slave.

2) Connect CH1 = SDA, CH2 = SCL, CH3 = RA0 (NACK), and CH4 = RA1 (timer diagnostic). Trigger on the rising edge of CH4.

3) Measure the interval between RA1 rising edges and compare it with your calculated timer interval.

4) With power off, remove one pull-up at a time and observe the effect. Restore both resistors and verify normal operation.

5) Configure 7-bit I²C decoding and capture START, the seven address bit (0x20), R/W=0, the ninth-clock NACK, and STOP. Verify RA0 indicates NACK. Note if your decoder displays the 8-bit address/R-W byte (0x40) instead and verify oscilloscope decode setup. Verify that RA1 rises before START and falls after STOP, with a clear idle gap between transactions.

6) With nominal pull ups installed, measure the SCL period/frequency, clock HIGH/LOW times, SDA/SCL rise times (30% to 70%), idle high HIGH and driven LOW voltages, and supply voltages.

7) Repeat the measurements for nominal, nominal ×10, nominal ÷2, and nominal ÷10 resistance targets. Use one selected standard resistance per line, with no combinations. Power off before changing resistors and measure each resistors actual value. Check electrical limits before powering each configuration. Calculate idle and LOW pull up currents from measured voltages and resistances. Compare predicted and measured values for both lines; note if idle current is below measurements resolution.

L6

TABLE 13-2: REGISTERS ASSOCIATED WITH SPI OPERATION

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Register on Page
INTCON	GIE/GIEH	PEIE/GIEL	T0IE	INTE	RBIE	T0IF	INTF	RBIF	32
PIE1	—	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	33
PIR1	—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	35
SSPBUF	Synchronous Serial Port Receive Buffer/Transmit Register								179
SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0	177
SSPSTAT	SMP	CKE	D/Ā	P	S	R/W̄	UA	BF	176
TRISA	TRISA7	TRISA6	TRISA5	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	40
TRISC	TRISC7	TRISC6	TRISC5	TRISC4	TRISC3	TRISC2	TRISC1	TRISC0	54

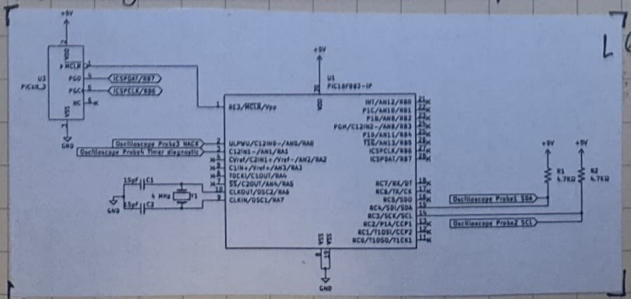
Legend: x = unknown, u = unchanged, – = unimplemented, read as '0'. Shaded cells are not used by the MSSP in SPI mode.

Note 1: Bit 6 of PORTA, LATA and TRISA are enabled in ECIO and RCIO Oscillator modes only. In all other oscillator modes, they are disabled and read '0'.

• Table 1: Registers Associated with SPI Operation

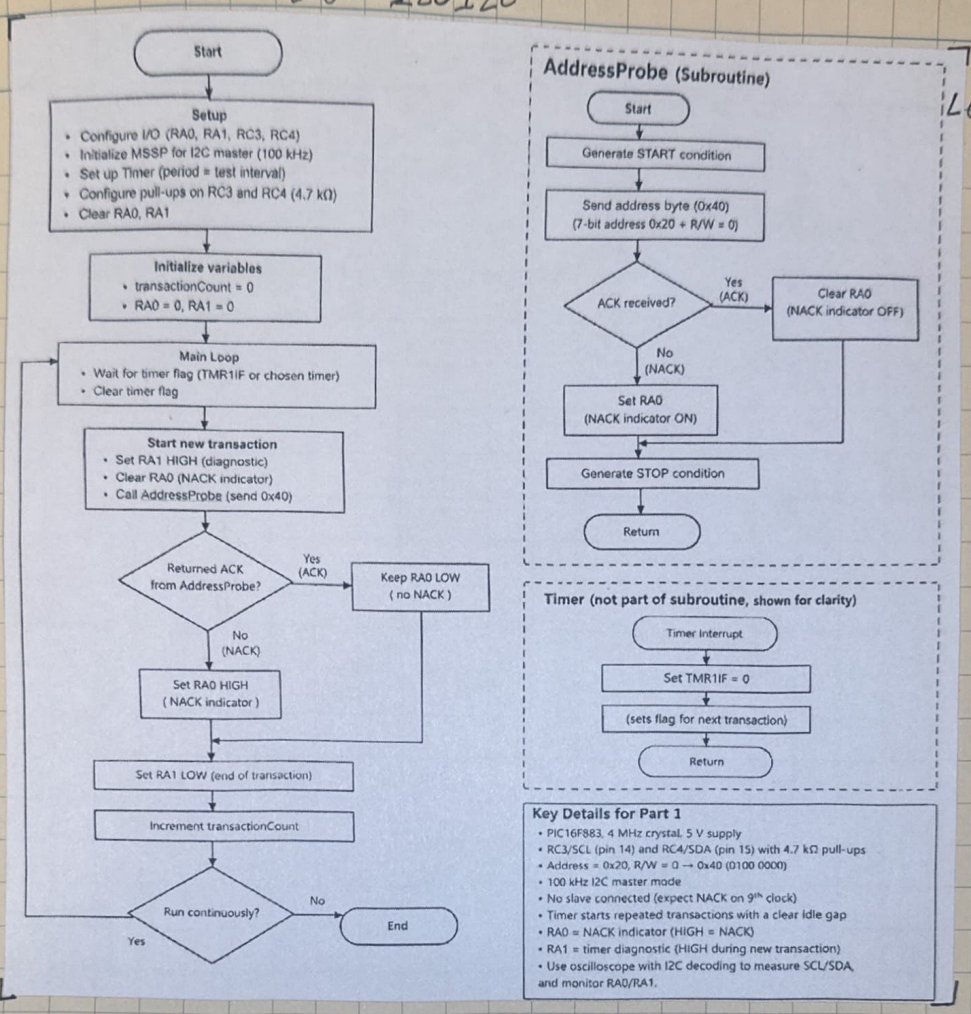
L6

• The next few pages will show the SFR's, flowchart, and operation code.



• Figure 1: I2C Master and Bus timing

10/26/2026
LRG



•Figure 2: Master and Bus timing for I2C flow chart for Part 1.

Calculated Target	*Standard Value*	*Resistor Rating*	*Predicted Electrical Results*	*Idle HIGH*	*Idle Current*	*LOW Current*	*Rise Time*
4.7 kΩ	4.7 kΩ	500mW	398.09 ns	5V	0mA	0.979 mA	0.398 µs
47 kΩ	47 kΩ	500mW	3.981 us	5V	0mA	0.098 mA	3.982 µs
2.35 kΩ	2.2 kΩ	500mW	186.34 ns	5V	0mA	2.091 mA	0.186 µs
470 Ω	470 Ω	500mW	39.809 ns	5V	0mA	9.787 mA	0.040 µs

The PIC16F883 datasheet spec for max output current for sink/source is 25mA, making each of the resistors in spec. However, the 47 kΩ resistor has the problem that it draws less current and produces a much slower rising edge, which exceeds the Standard-mode timing limit under the 100 pF assumption. The 470 Ω resistor finds the problem of power, which can be less effective when using things like batteries.

•Table 2: Predicted and target values for timing and current for voltage and current ratings on buss.
- The reason 4.7kΩ is the best is because it has the lowest of the current draw and the best rise time. The higher resistor 47kΩ has the lowest current draw but terrible Rise Time. The 2.35kΩ resistor is good but the current draw is still very high and would drain power fast. The 470Ω is the same even if the rise time is better the current draw is to much.

```
Setup:
    ;       Bank 3
    BANKSEL ANSEL
    CLRF    ANSEL           ;make every pin a normal digital pin
    CLRF    ANSELH          ;RB0 and RB1 can also be analog pins, so this matters
    ;       Bank 1
    BANKSEL TRISA
    BCF     TRISA,0         ;RA0 is an output (NACK indicator)
    BCF     TRISA,1         ;RA1 is an output (timer diagnostic)
    BSF     TRISC,3         ;RC3 is an input for SCL (for I2C)
    BSF     TRISC,4         ;RC4 is an input for SDA (for I2C)
    MOVLW   9
    MOVWF   SSPADD          ;100k Hz clock
    MOVLW   0b10000000
    MOVWF   SSPSTAT         ;set the Sample bit high for normal speed mode at 100k Hz
    CLRF    SSPCON2         ;set the register low so no START/STOP is running
    MOVLW   249
    MOVWF   PR2             ;Timer2 counts 250 steps
    ;       Bank 0
    BANKSEL PORTA
    CLRF    PORTA           ;both markers start LOW
    CLRF    PORTC           ;display starts blank
    MOVLW   0b00101000
    MOVWF   SSPCON          ;set I2C on master mode and Enable the serial port pins SDA and SCL
    MOVLW   0b01001101
    MOVWF   T2CON           ;turn Timer2 ON, flag every 10ms
; MAIN LOOP one address probe every 10 ms
Main:
    BTFSS   PIR1,1          ;has the timer flag (TMR2IF) been set?
    GOTO    Main            ;no - keep waiting
    BCF     PIR1,1          ;yes - clear the flag

    BSF     PORTA,1         ;RA1 HIGH: a new transaction starts
    CALL    AddressProbe    ;START, address, check ACK, STOP
    BCF     PORTA,1         ;RA1 LOW: the STOP is finished
    GOTO    Main

; AddressProbe send START, the address byte, look at ACK, send STOP.
AddressProbe:
    BANKSEL PORTA
    BCF     PORTA,0         ;clear the NACK indicator

    BANKSEL SSPCON2
    BSF     SSPCON2,0       ;SEN = 1: generate a START
WaitStart:
    BTFSC   SSPCON2,0       ;SEN goes back to 0 when START is done
    GOTO    WaitStart

    BANKSEL PIR1
    BCF     PIR1,3          ;clear SSPIF (START also sets it)
    MOVLW   SlaveAddress
    MOVWF   SSPBUF          ;sending this byte starts the 9 clocks
WaitSend:
    BTFSS   PIR1,3          ;SSPIF = 1 after the 9th clock
    GOTO    WaitSend
    BCF     PIR1,3          ;clear it again

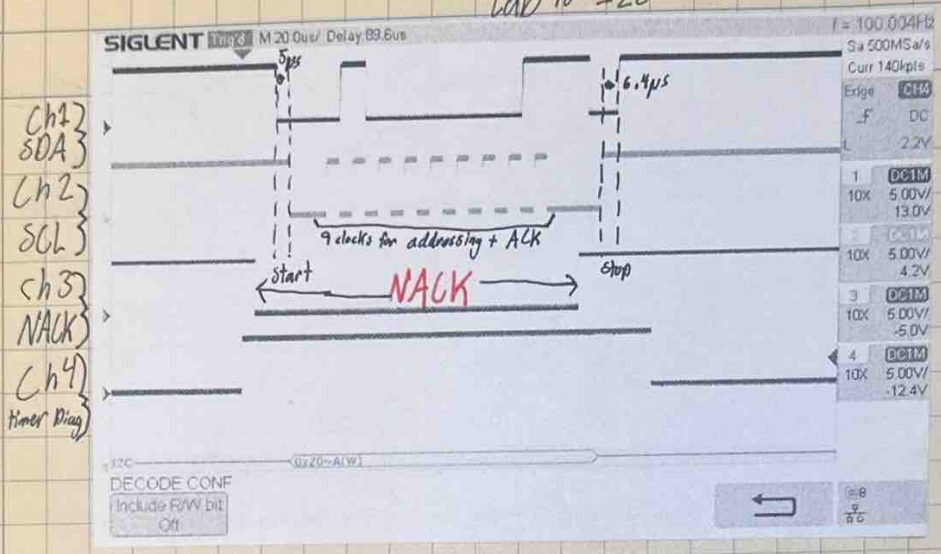
    BANKSEL SSPCON2
    BTFSS   SSPCON2,6       ;ACKSTAT: 0 = ACK, 1 = NACK
    GOTO    SendStop        ;ACK - go straight to STOP
    BANKSEL PORTA
    BSF     PORTA,0         ;NACK - turn the RA0 indicator ON
    BANKSEL SSPCON2

SendStop:
    BSF     SSPCON2,2       ;PEN = 1: generate a STOP
WaitStop:
    BTFSC   SSPCON2,2       ;PEN goes back to 0 when STOP is done
    GOTO    WaitStop

    BANKSEL PORTA           ;back to bank 0
    RETURN

    End
```

•Figure 3: Setup with SFR and main operation Master / Bus timing (I^2C)



•~~AV~~ LG Figure 4: Oscilloscope Capture of Address sent via I2C of 0x20 with R/W off bit

- Addressing-only with no slave sending an address of 0x20 with the R/W bit turned off on the oscilloscope. The trigger was set to the rising edge of RA1 channel 4. CH1=SDA, CH2=SCL, CH3=Nack (RA0), CH4=Timer diagnostic (RA1). With 4.7K Ω pull up resistors which work best for rise time and power consumption. Above shows the start and stop bits. The address sent was a hex 40 but in binary on SDA it sees a (0100000) for the address with a 0 for the write. START: SDA falls whe SCL is still high and falls shortly after. STOP the SDA is pulled low while SCL is "rises high. A rising SDA with SCL High is the STOP condition.

TJR
10-27-26

•Part 2 - I2C Master with an ADS1115 Sensor Slave

•Goal: Use the Part 1 I2C master to identify an ADS1115, inspect and change its configuration, and read its ADC results. Observe the conversion-ready signal to choose a suitable reading interval.

•~~Before Lab~~: LG

•Before Lab:

- Bring your working Part 1 address-probe program, compiled and programmed into the PIC.
- Skim the ADS1115 data sheet. Find the PIN descriptions, supply/inputs limits, and I2C address selection. You will investigate the registers and configure the ADC as you work through the Lab.

•Suggested implementation (optional): The three-levels I^2C software design guide offers one possible implementation with flowcharts, pseudocode, and I^2C transaction sequences. It is a reference, not required program structure. You may adopt it or develop your own implementation that meets the lab requirements; no particular routine layout or number of source files is required.

•In the Lab:

1) Find the ADS1115 address. Connect the breakout to the Part 1 SDA/SCL bus. Before applying power, check the board's supply and I2C logic levels against the PIC circuit. Document the connections in your schematic.

* Pull-up hint: With LG with power off and the breakout disconnected from the PIC and external pull-ups, measure resistance with a DMM from SDA and SCL to the board's supply rail. Check ADDR and ALERT/RDY the same way. These measurements can reveal installed pull-ups; inspect the board markings or schematic to confirm because other onboard components may affect an in-circuit resistance reading.

10/27/2026
LRG

Lab 10 - I²C

•Find the four possible 7-bit ADS1115 addresses in the data sheet. With ADDR tied to ground (GND), use the Part 1 probe to try all four addresses, recording ACK or NACK for each. Power off before moving ADDR to VDD, then probe all four addresses again. Modify the probe so each trial runs once after MCLR reset then enters a do-nothing loop instead of repeating. For each scope capture, arm the oscilloscope in Single Sequence mode, then press and release MCLR.

- For each candidate: START → [address + Write bit] → ACK or NACK → STOP

7-bit address from data sheet	Transmitted write byte	ADDR = GND	ADDR = VDD
1001000b	0	ACK / NACK	ACK / NACK
1001001b	0	ACK / NACK	ACK / NACK
1001010b	0	ACK / NACK	ACK / NACK
1001011b	0	ACK / NACK	ACK / NACK

•Table 3: Record all results here for addresses and Transmitted bytes

- Save a decoded ACK capture for each ADDR condition. For the remaining steps, choose either ADDR connection that produced an ACK and use its confirmed address. In the sequences below, ADDR+W and ADDR+R mean the transmitted address byte formed from your chosen 7-bit address with the write bit (0) and or read bit (1).

2) Read the Config register. Find the ADS1115 register-pointer table. The one byte pointer 0x01 selects Config; it does not overwrite Config. On a freshly powered ADS1115, the expected Config value is 0x8583. Select the pointer, then read the 16-bit register with a new start.

* Select config (once):
  START → ADDR+W → ACK → 0x01 → ACK → STOP
* Read config (once):
  START → ADDR+R → ACK → Config MSB → PIC ACK → Config LSB → PIC NACK → STOP

-Run the pointer write and Config read once after MCLR reset, then enter a do nothing loop. Arm the oscilloscope in Single Sequence mode before releasing MCLR. Combine the two bytes and explain the observed Config fields using the data sheet. MCLR resets the PIC, not the ADS1115; to check the ADS1115's power-up value; power cycle the ADC before this test. The ACKs after address/pointer bytes come from the ADS1115, PIC ACK and PIC NACK are sent by the master after received bytes.

3) Configure the ADC. Now use the Configure register descriptions to determine the two bytes required for:

- AIN0 relative to GND
- ±6.144V full-scale range
- Continuous conversion mode
- 128 samples per second
- Comparator disabled.

-Document each chosen field and derive the complete 16-bit value. Write it using this transaction.

* Configure ADC (once at startup):
  START → ADDR+W → ACK → 0x01 → ACK → [Config MSB] → ACK → [Config LSB] → ACK → STOP

10/27/2026
LRG

- Run the Config write once after MCLR reset, then enter a do nothing loop for its Single Sequence capture. Read Config again to verify the setting, also once, using the read transaction from Step 2; the write already selected Config. Do not repeat the Config write on every ADC read.

4) Read ADC conversions. Connect a potentiometer across ADS1115 VDD and GND, with its wiper at AIN0. Don't exceed the ADC's supply voltage at its analog input. The ADS1115 now converts continuously on its own. The PIC reads the latest completed result only when it requests it. If the ADS1115 has lost power since Step 3, configure it once before starting the read loop. Start with the timer interval from Part 1; you will choose a final interval in Step 5. Each PIC polling cycle repeats both transactions:

★ Select Conversion:
START → ADDR+W → ACK → 0x00 → ACK → STOP

★ Read Conversion:
START → ADDR+R → ACK → ADC MSB → PIC ACK → ADC LSB → PIC NACK → STOP

★ Repeat both transactions at the PIC timer interval.

- The ADS1115 retains its register pointer, but for this Lab send pointer 0x00 on every polling cycle. If an address or write-data byte is NACKed, stop sending data, attempt STOP, and preserve/report the error. On a timeout or bus-low error, report the failure and don't assume STOP or bus recovery will succeed; don't continue the transaction. The master's NACK after the final received byte is normal. Combine the ADC bytes as a signed 16-bit result. For the assigned full-scale range:

$$ADC_{16} = (MSB \ll 8) \mid LSB$$

$$V_{AIN0} = \frac{\text{signed } ADC_{16}}{32768}(6.144V)$$

★ For example, a positive result of 0x2000 = 8192 counts gives:

$$V_{AIN0} = \frac{8192}{32768}(6.144v) = 1.536V$$

- The ±6.144v range determines ADC scaling; it does not raise the allowable input voltage above ADS1115 VDD.

5) Observe ALERT/RDY and choose a polling interval. Find Conversion Ready Pin in the ADS1115 data sheet. Configure ALERT/RDY for conversion-ready pulses using the threshold registers and Config fields. The output is open drain; ensure it has a pull-up to a compatible supply.

★ Set low threshold (once):
START → ADDR+W → ACK → 0x02 → ACK → [Lo MSB] → ACK → [Lo LSB] → ACK → STOP & STOP

★ Set high threshold (once):
START → ADDR+W → ACK → 0x03 → ACK → [Hi MSB] → ACK → [Hi MSB] → ACK → STOP

★ Enable conversion-ready output (once):
START → ADDR+W → ACK → 0x01 → ACK → [Updated Config MSB] → ACK → [Updated Config LSB] → ACK → STOP

- Perform the ALERT/RDY setup writes only once, then leave the PIC in a do-nothing loop while the ADS1115 converts on its own. Trigger the oscilloscope on ALERT/RDY. Measure the interval between pulses and calculate the observed conversion rate. Compare it with the configured data rate. Choose and justify a PIC timer interval for repeated I2C reads. Explain whether that interval will retrieve every conversion or skip some. Confirm the actual interval between I²C read cycles on the scope.

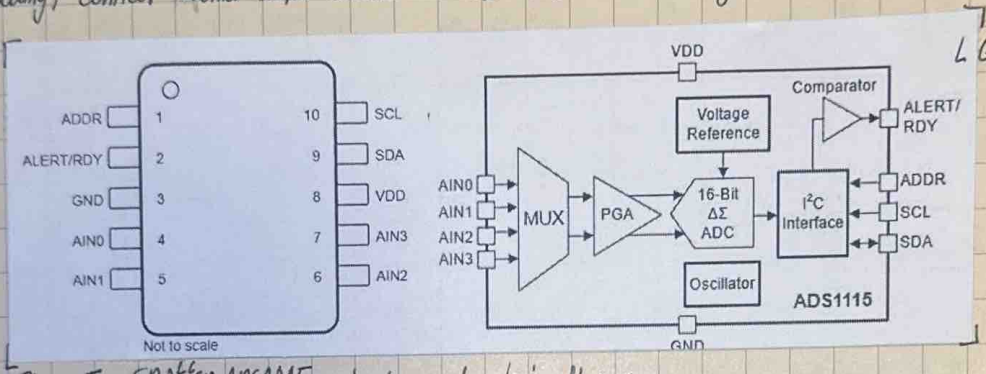
10/28/2026
LRG

Lab 10 - I²C

10/28/2026
LRG

6) Capture continuous reading. With your selected PIC timer interval, capture multiple consecutive Conversion-pointer/write and conversion-read transactions at each potentiometer extreme. Measure AIN0 with a DMM at each setting and compare the measured voltage with the ADC result. Use CH1=SDA and CH2=SCL for decoding; connect another scope channel to ALERT/RDY for the timing test.

LG



•Figure 5: SDA & SD ADS1115 pinout and logic diagram

LG

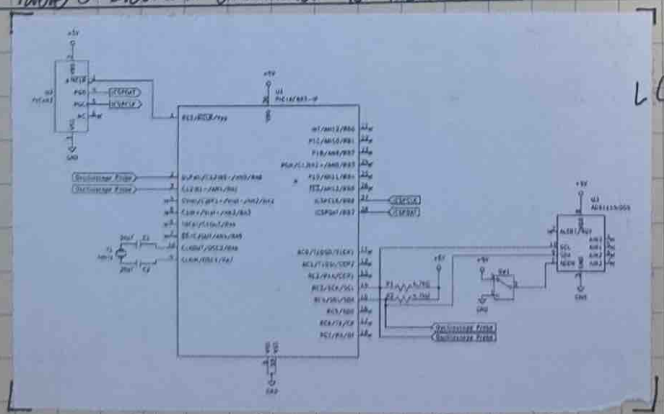
ADDR PIN CONNECTION	TARGET ADDRESS
GND	1001000b
VDD	1001001b
SDA	1001010b
SCL	1001011b

•Table 4: Addressing targes with pin Connection to ADDR.

		MIN	MAX	UNIT
Power-supply voltage	VDD to GND	–0.3	7	V
Analog input voltage	AIN0, AIN1, AIN2, AIN3	GND – 0.3	VDD + 0.3	V
Digital input voltage	SDA, SCL, ADDR, ALERT/RDY	GND – 0.3	5.5	V
Input current, continuous	Any pin except power supply pins	–10	10	mA
Temperature	Operating ambient, T_A	–40	125	°C
	Junction, T_J	–40	150	
	Storage, T_{stg}	–60	150	

LG

•Table 5: Electrical characteristics for the ADS1115

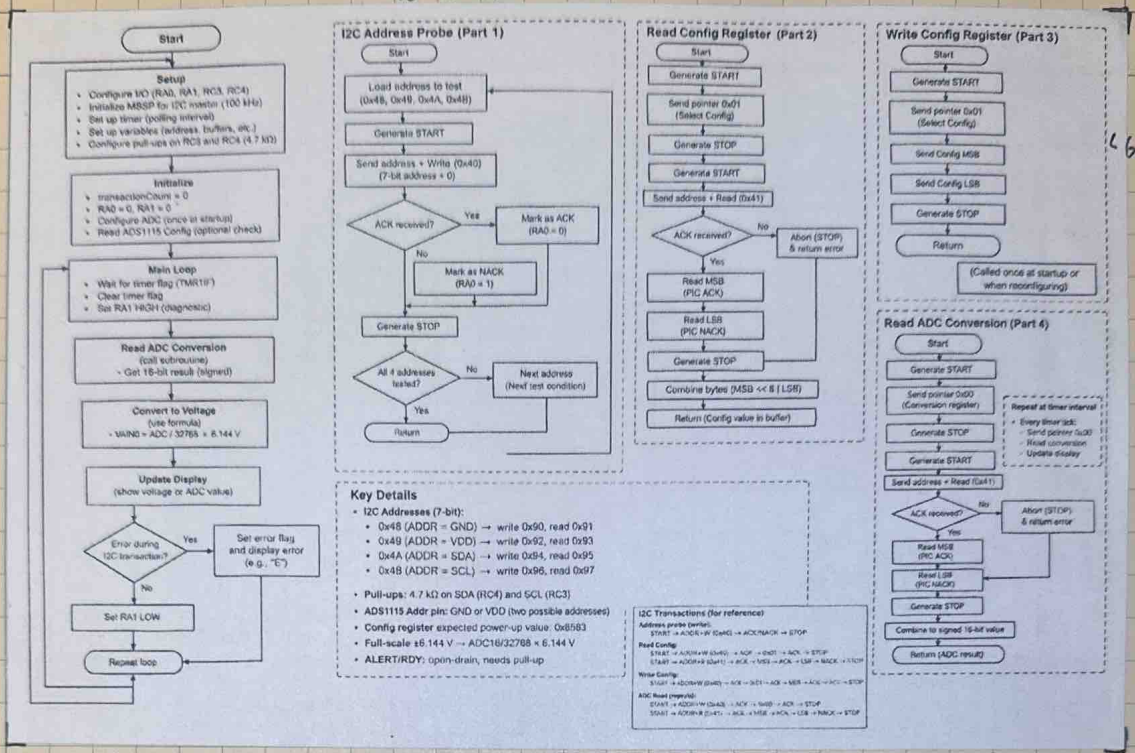


LG

•Figure 6: Full schematic for Part 2 ADS1115 Slave Sensor

LG

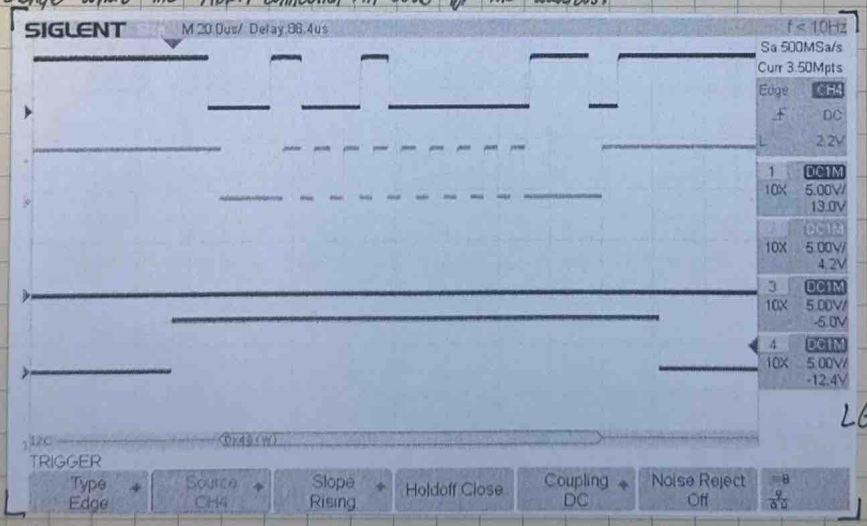
10/28/2026
LRG



LG

•Figure 7: Full flow chart for all 5 tests, to observe I²C transmission with slave board

-The next captures show the recored address and the acknowledge. This was proven by having to manual change where the ADDR connection Pin was for the address.



LG

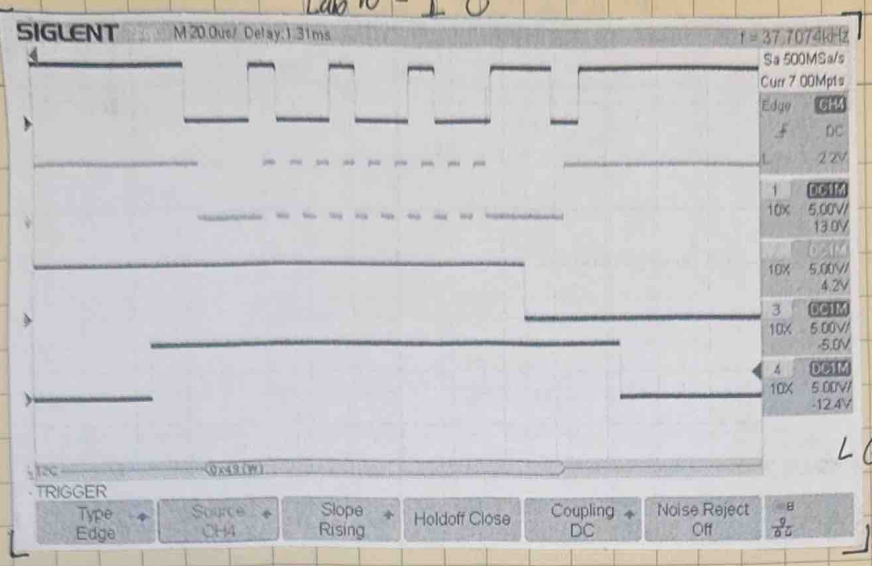
•Figure 8: Waveform capture for acknowledge for ADDR to GND (written byte = 0x90 for with R/W=0 = 0x48)

-The oscilloscope capture shows the I2C value = 0x48 (W) which means an ACK happened, if no acknowledge it would show = 0x48 ~A(W). The next few captures show the acknowledged pin connections for which the ADDR is connected.

LG

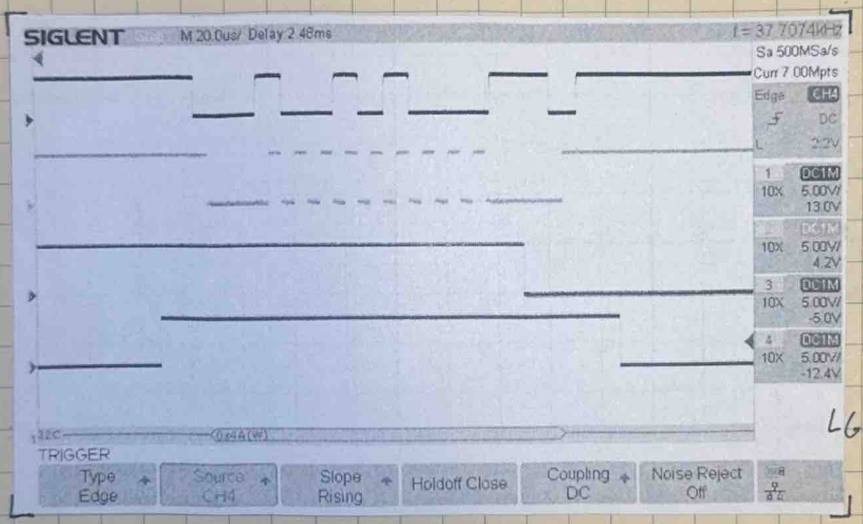
10/29/2026
LRG

Lab 10 - I²C



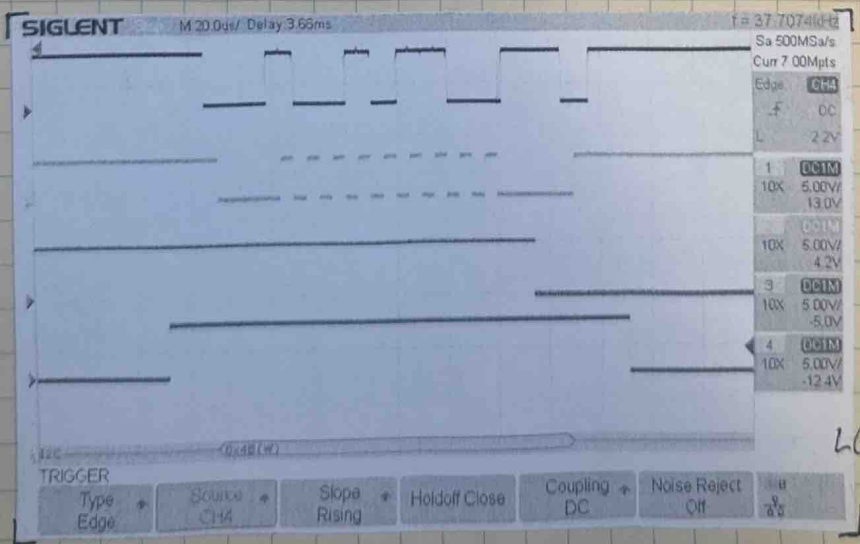
LG

- Figure 9: Waveform capture of acknowledge that ADDR connected to VDD (written byte 0x92 meaning 0x49 with R/W=0)



LG

- Figure 10: Waveform capture of acknowledge that ADDR connection to SDA (written byte 0x94 meaning 0x4A with W/R=0)

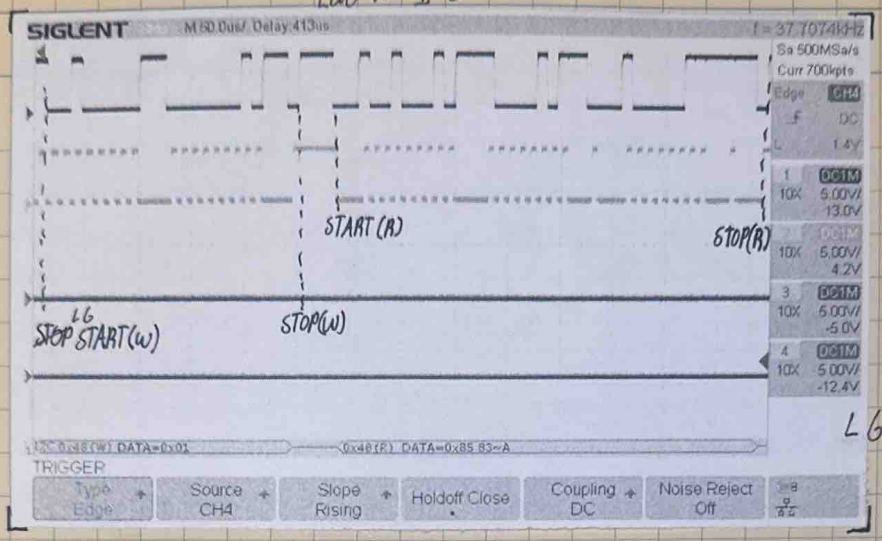


LG

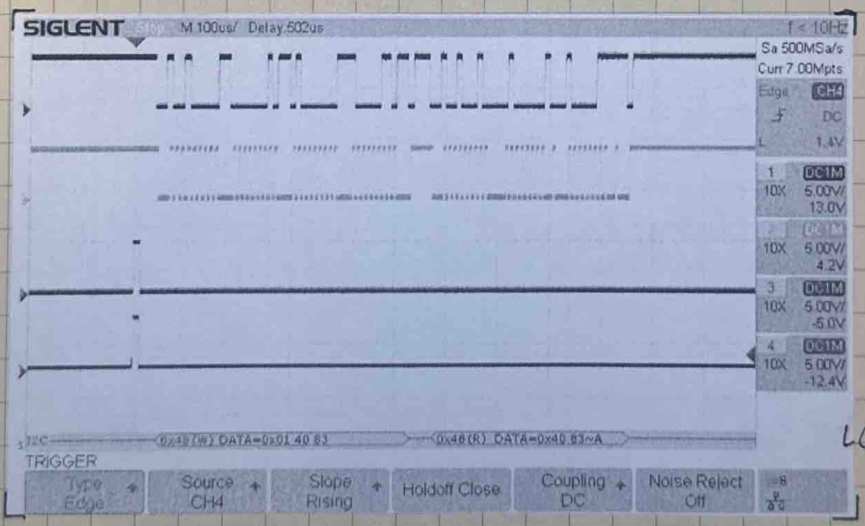
- Figure 11: Waveform capture of acknowledge that ADDR connection to SCL (written byte 0x96 meaning 0x4B with W/R=0)

- All of these captures prove step 1 was done. For step 2 we wanted to read the Config register and reseve an value of 0x8583 from the ADS1115. By writing the one byte pointer 0x01 which selects the Config.

10/29/2026
LRG



- Figure 12: Waveform capture of Sent 0x01 Data and received 0x8583 Data with NACK
- The process goes as follows:
  - ★ START → ADDR+W → ACK → 0x01 → ACK → STOP
  - ★ START → ADDR+R → ACK → Config MSB → PIC ACK → Config LSB → PIC NACK → STOP
- This was how the code process works to send the slave what it needs inorder to send the Config bytes back. This one capture proves part 2 LG step 2 in I^2C communications.



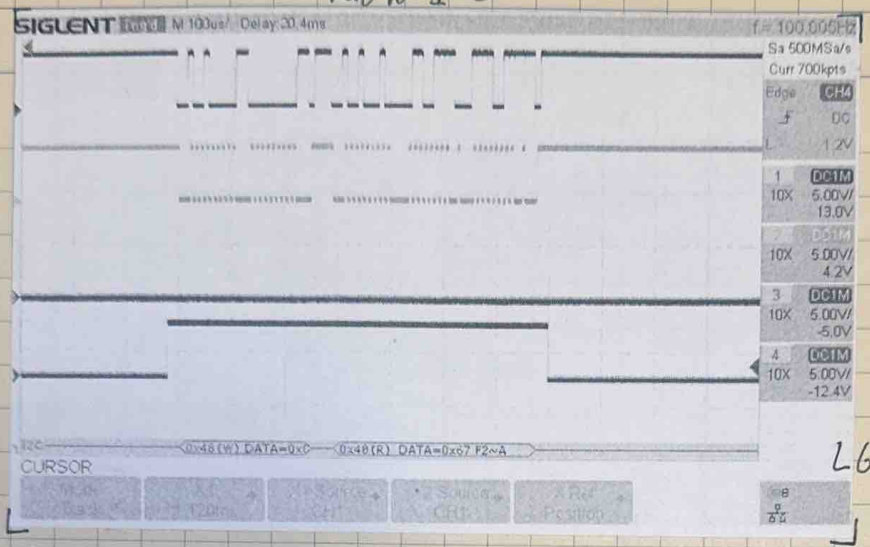
- Figure 13: Waveform capture of Captured ADC config without a pot
- The processed followed was:
  - ★ START → ADDR+W → ACK → 0x01 → ACK → [Config MSB] → ACK → [Config LSB] → ACK → STOP
- This runs 1 time and then enters a do nothing loop.

LG

Lab 10 - I²C

10/29/2026

LRG



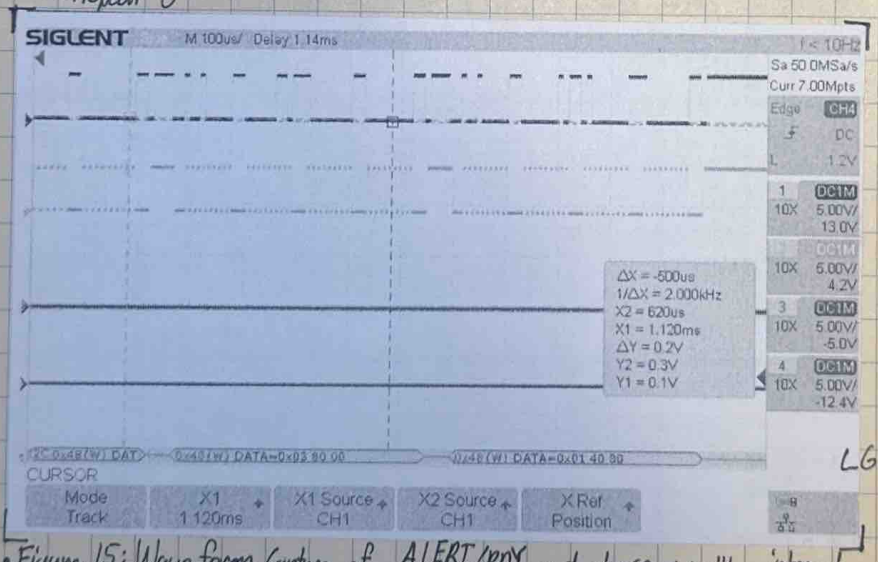
- Figure 14: Waveform capture of Reading ADC conversions and reading different values (Pot voltage = 4.992v)

-This is a capture of use writing to the slave module and requesting ADC values back and these were the followed steps.

★ START → ADDR+W → ACK → 0x00 → ACK → STOP

★ START → ADDR+R → ACK → ADC MSB → PIC ACK → ADC LSB → PIC NACK → STOP

★ Repeat ☝



- Figure 15: Wave form Capture of ALERT/RDY and choose a polling interval

-This was meant to setup the code for pointing Address register and configuring the set 16 conversion ready pulses and the threshold registers and config fields.

★ START → ADDR+W → ACK → 0x02 → ACK → [Lo MSB] → ACK → [Lo LSB] → ACK → STOP

★ START → ADDR+W → ACK → 0x03 → ACK → [Hi MSB] → ACK → [Hi LSB] → ACK → STOP

★ START → ADDR+W → ACK → 0x01 → ACK → [Update Config MSB] → ACK → [Update Config LSB] → ACK → STOP

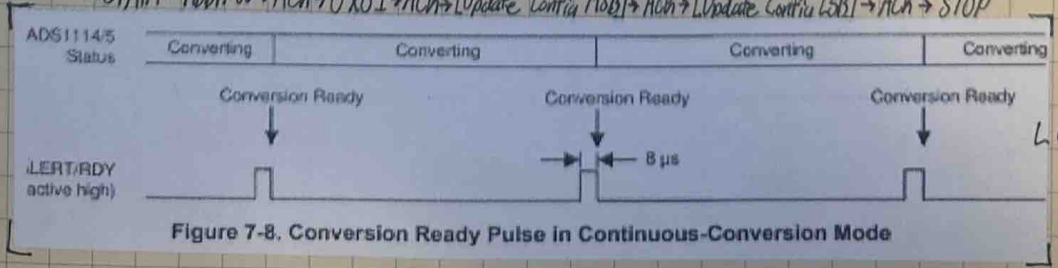
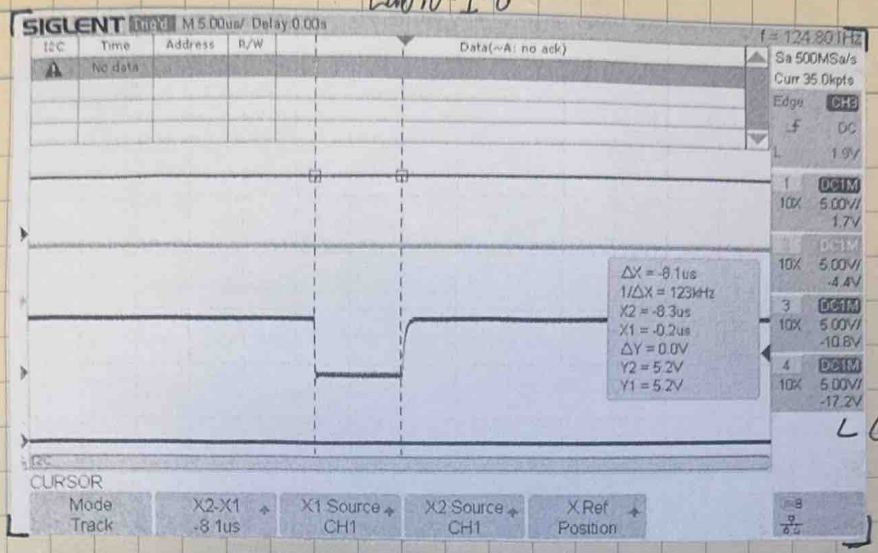


Figure 7-8. Conversion Ready Pulse in Continuous-Conversion Mode

- Figure 16: Conversion Ready Pulse in Continuous-Conversion Mode for Part 6

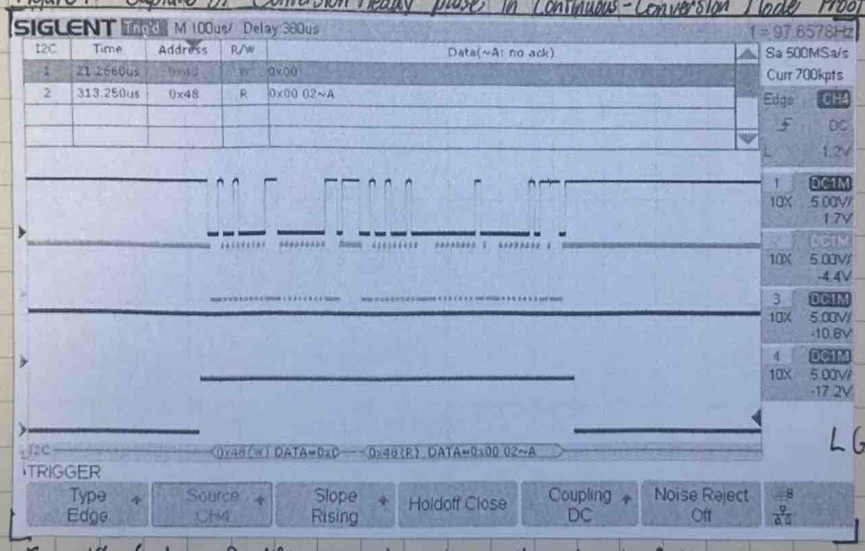
10/29/2026

LRG



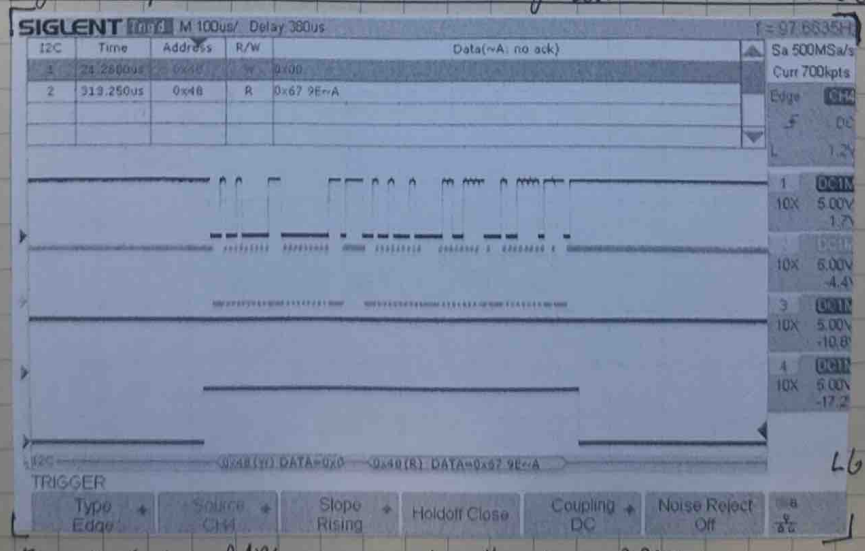
LG

• Figure 17: Capture of Conversion Ready pulse in Continuous-Conversion Mode Proof with measured time.



LG

• Figure 18: Capture of ADC conversion at a voltage level of DMM measured = 0.000v.



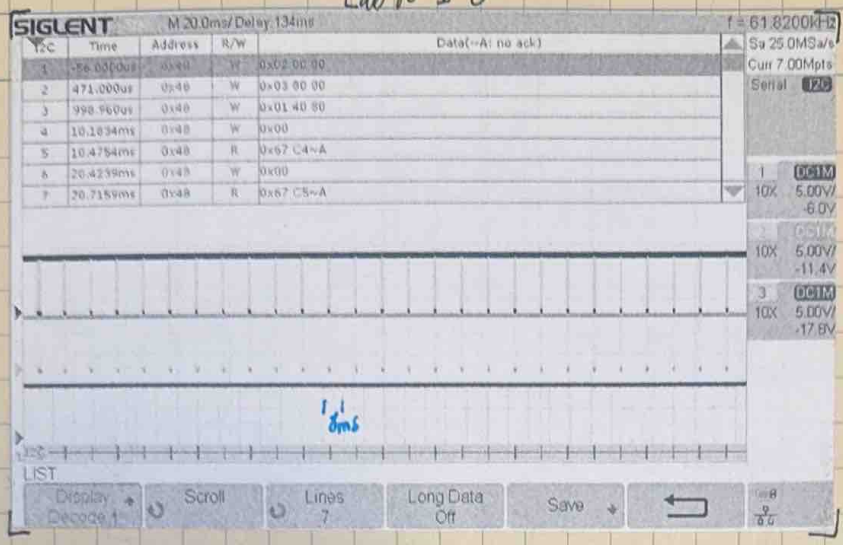
LG

• Figure 19: Capture of ADC conversion at a voltage level of DMM measured 4.992v

10/29/2026

LRG

Lab 10 - I²C



- Figure 20: Waveform capture of start up to continous reading with Setup.

Figure 8-1. Address Pointer Register

7	6	5	4	3	2	1	0
RESERVED						P[1:0]	
W-000000b						W-00b	

LEGEND: R/W = Read/Write; R = Read only; W = Write only; -n = value after reset

Table 8-1. Address Pointer Register Field Descriptions

Bit	Field	Type	Reset	Description
7:2	RESERVED	W	000000b	Always write 000000b
1:0	P[1:0]	W	00b	**Register address pointer** 00b : Conversion register 01b : Config register 10b : Lo_thresh register 11b : Hi_thresh register

- Figure 21: Address Pointer Register and Descriptions for ADS1115

I2C	Time	Address	R/W	Data
1	-56.0000u	0x48	W	0x02 00 00
2	471.000us	0x48	W	0x03 80 00
3	998.960us	0x48	W	0x01 40 80
4	10.1834m:	0x48	W	0x00
5	10.4754m:	0x48	R	0x67 C4~A
6	20.4239m:	0x48	W	0x00
7	20.7159m:	0x48	R	0x67 C5~A
8	30.6613m:	0x48	W	0x00
9	30.9533m:	0x48	R	0x67 C5~A
10	40.9018m:	0x48	W	0x00
11	41.1938m:	0x48	R	0x67 C3~A
12	51.1422m:	0x48	W	0x00
13	51.4342m:	0x48	R	0x67 C2~A
14	61.3796m:	0x48	W	0x00
15	61.6716m:	0x48	R	0x67 C3~A
16	71.6201m:	0x48	W	0x00
17	71.9120m:	0x48	R	0x67 C3~A

TSR 10-29-26

- Table 6: Table from measured values to startup Continous reading as well as ADC Readings.

- For link to full code see the following link:
  - https://github.com/groelke/Lab_10_I2C/tree/main/LAB_10_Part_2_I2C.X

- See the next page for GPRs and SFR's were the same from part 1 so it includes the continus startup reading code.

10/29/2026
LRG

10/29/2026
LRG

Lab 10 - I2C

```
;ADS1115 address bytes (7-bit address shifted left, R/W = 0)
   AddressGND      EQU 0x90      ;ADDR pin on GND  0x48
   AddressVDD      EQU 0x92      ;ADDR pin on VDD  0x49
   AddressSDA      EQU 0x94      ;ADDR pin on SDA  0x4A
   AddressSCL      EQU 0x96      ;ADDR pin on SCL  0x4B

;The address used for steps 2 to 5. This is ment for after Part 1
   AddressWrite    EQU AddressGND        ;address + write bit
   AddressRead     EQU AddressGND + 1    ;address + read bit

   RegisterPointer EQU 0x70    ;which ADS1115 register to select (0x00 to 0x03)
   WriteHighByte   EQU 0x71    ;high byte to write
   WriteLowByte    EQU 0x72    ;low byte to write
   ReadHighByte    EQU 0x73    ;high byte we read back
   ReadLowByte     EQU 0x74    ;low byte we read back
; STEP 4 - configure once, then read the conversion every timer tick.
; The answer is in ReadHighByte and ReadLowByte (signed 16-bit).
Step4_Poll:
    MOVLW   0x02
    MOVWF   RegisterPointer     ;Lo threshold register
    CLRF    WriteHighByte
    CLRF    WriteLowByte        ;Lo = 0x0000
    CALL    WriteRegister
    MOVLW   0x03
    MOVWF   RegisterPointer     ;Hi threshold register
    MOVLW   0b10000000
    MOVWF   WriteHighByte
    CLRF    WriteLowByte        ;Hi = 0x8000
    CALL    WriteRegister
    MOVLW   0x01
    MOVWF   RegisterPointer     ;Config register
    MOVLW   0b01000000
    MOVWF   WriteHighByte
    MOVLW   0b10000000
    MOVWF   WriteLowByte        ;same Config, comparator queue = 00
    CALL    WriteRegiste
PollLoop:
    BTFSS   PIR1,1              ;timer flag?
    GOTO    PollLoop
    BCF     PIR1,1              ;clear it
    BSF     PORTA,1             ;RA1 HIGH: a cycle starts
    CLRF    RegisterPointer     ;0x00 = Conversion register
    CALL    SetPointer          ;START, ADDR+W, 0x00, STOP
    CALL    ReadTwoBytes        ;START, ADDR+R, high, low, STOP
    BCF     PORTA,1             ;RA1 LOW: cycle finished
    GOTO    PollLoop
```

LG

*Figure 22: GPR's and the countinuous reading setup code.

LG

10/29/2026
LRG

10/30/2026
LRG

Lab 10 - I²C

Part 3 - PIC I²C Slave: Inputs, Display, and ADC

Goal: Part 3 requires two PIC16F883 programs: I²C master code and I²C slave code, running on separate PICs. Develop the slave at 7-bit address 0x20 and extend your part 2 master code to test the slave, read its digital inputs, write display characters, and retrieve 10-bit ADC measurements. The slave uses a small register-pointer interface similar in concepts to the ADS1115. The slave must normally service I²C events promptly. Either interrupts or polling the MSSP event flag is acceptable. Keep hardware communication routines reusable and separate from application behavior.

Before Lab:

- Bring working Part 1 & Part 2 master code. For Part 3, create a master MPLAB X project and a separate slave MPLAB X project. Writing, building, programing, and testing both master and slave firmware throughout Part 3. Select the correct Main Project before programming.
- Draw the complete two pic system schematic: RC3/SCL (pin18), RC4/SDA (pin 15), common ground, and the effective SDA/SCL pullups. Account for pull ups already installed on the "breakout" boards; don't assume every device needs another pair.
- Use the PIC16F883 data sheet to document MSSP 7-bit slave mode setup, addressing "matching", SSPBUF, ACK/NACK, receive/transmit events, SSPIF, CKP/clock stretching, SSPOV, and WCOL. Explain which device controls the final ACK/NACK of a read and when the slave can hold SCL LOW.
- Prepare flowcharts for slave initialization and servicing an I²C event. Include receive-address, receive-data, transmit-data, and error paths. Extend these flowcharts as new behavior is introduced.

- Suggested software organization (optional): One approach is to separate the slave code into Level 1 (MSSP hardware events), level 2 (pointer and read/write transactions), and Level 3 (PORTB, display, ADC). The master code can reuse Part 2 I²C routines, with new application logic to send each test transaction and receive its results. This is a suggested design, not a required file layout. Each PIC project may use one PIC-AS .S file or multiple .S modules if responsibilities are clear. Don't put executable routines in .inc files.

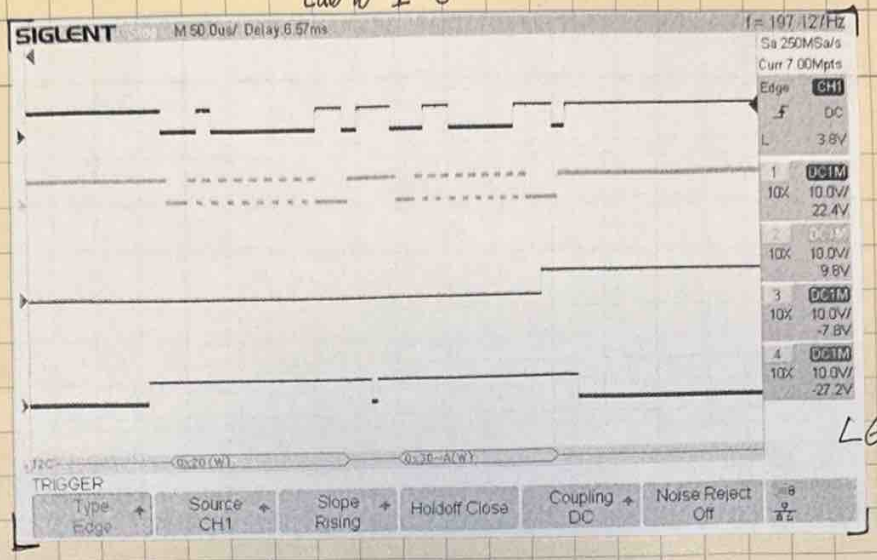
In the Lab: For every step, develop and test both sides of the I²C link: the master must initiate the required transactions and process any returned data or status; the slave must recognize its address and handle the corresponding requests. Don't submit only the slave ~~software~~ "firmware" or rely on unchanged Part 2 master for the completed Part 3 assignment.

1) Verify the slave address. Assemble the minimum slave circuit and the connection "it" to the existing master SDA/SCL bus. Confirm the slave PIC is programed and powered. Configure its MSSP for the 7-bit address 0x20. Use the Part 1 master address probe, first with the assigned address and then with an unrelated address. Capture an ACK for the correct Address and a NACK for the other address. The 7 bit address and complete transmitted bytes are different:

```
START → 0x20 + W (0x40) → slave ACK → STOP
START → other address + W → NACK → STOP
```

Figure 23: Program pattern for Step 1.

11/2/2026
LRG



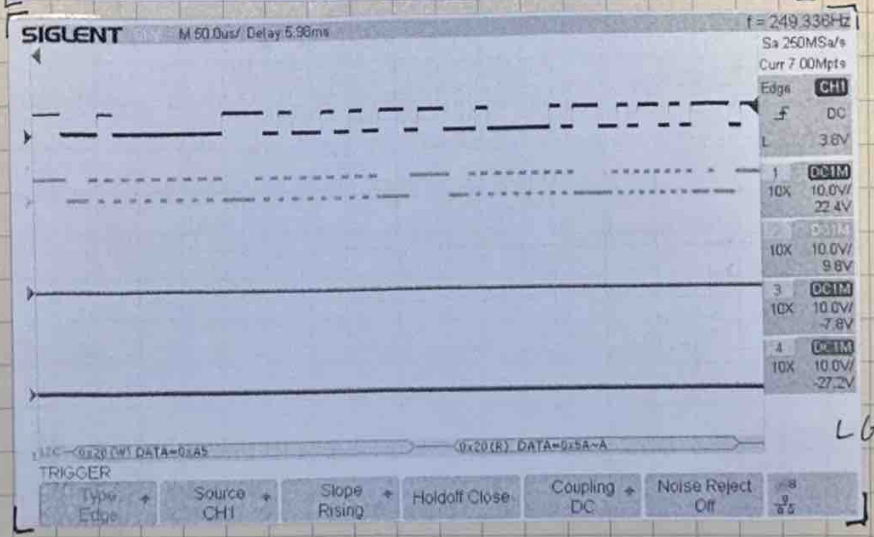
LG

- Figure 23: Write correct address to verify connection and an incorrect address to verify operation.

2) Test one recieved byte send one transmitted byte. Extend the slave to accept a byte after its address and modify the master to perform both tests. First write 0xA5 and check both slave ACK's. Next have the slave return a fixed byte 0x5A to a Master read. For the one-byte read, the master sends NACK after receiving the byte.

```
Write test: START → 0x40 → ACK → 0xA5 → ACK → STOP
Read test:  START → 0x41 → ACK → 0x5A → PIC NACK → STOP
```

LG



LG

- Figure 24: Write and Read tests to verify both test of transmitted and recevid bytes

3) Add the register-pointer interface. A slave write begins with a pointer byte. The pointer selects a logical register; optional following bytes write the register. A later master read returns data from the selected register. Retain the selected pointer after STOP sequences. For the inital diagnostic, temporarily echo the selected pointer once read. Test at least two different values such as 0xA5 and 0x3C. Then implement the following application register map.

Pointer	Function	Direction	Data
0x00	PORTB switches	Read	One byte, RB5:RB0
0x01	DLG7137 display	Write	One 7-bit ASCII character
0x02	PIC ADC at AN2	Read	Two bytes, high then low
0x04	Device status, including I²C status	Read	One byte

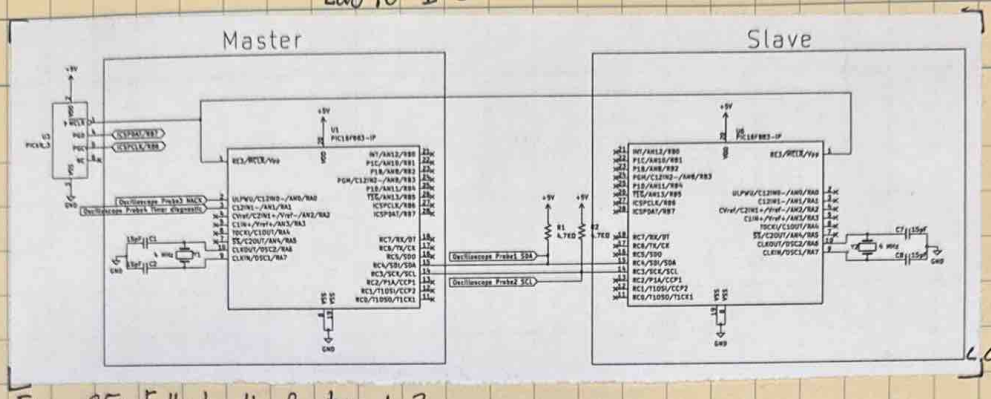
LG

- Table 7: Pointers and there functions

-Document how code handlers an unsupported pointer or an unexpected number of data bytes. - STEP 4

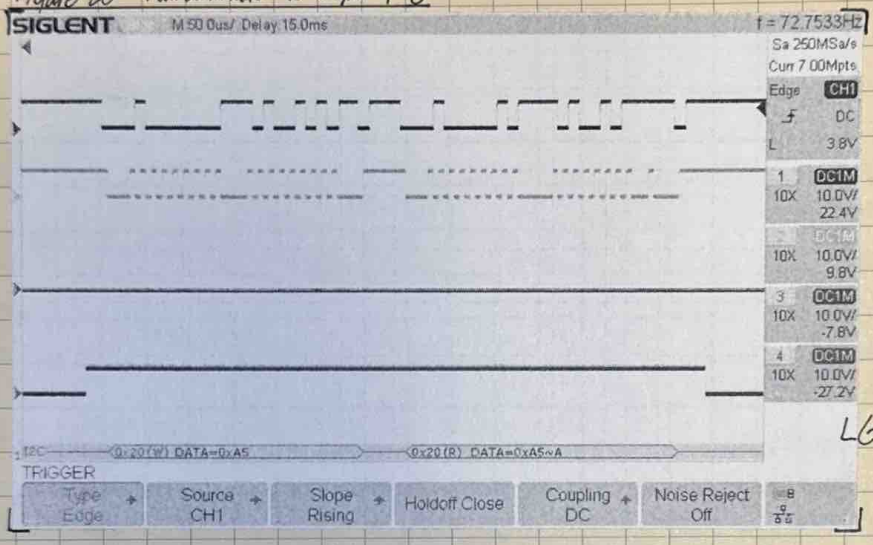
Lab 10 - I²C

11/12/2026
LRG



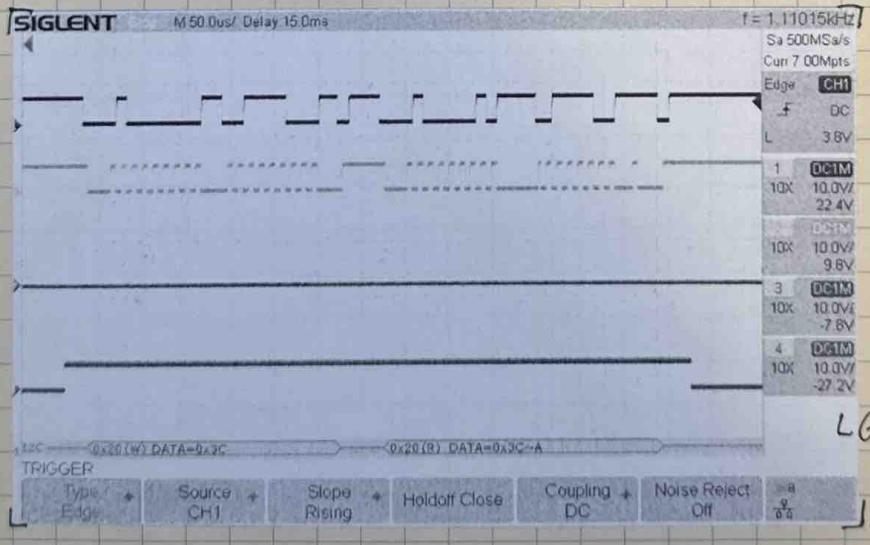
LG

•Figure 25: Full schematic fo steps 1-3



LG

•Figure 26: Pointer selects logical register for testing 0xA5 master to slave



LG

•Figure 27: Pointer selects logical register for testing 0x36 master to slave pointer.

4) Read switches and status. Connect six dip DIP switches from slave RB0-RB5 to Ground. Enable suitable pull-ups so an open switch reads 1 and a closed switch reads 0. Reserve RB6/RB7 for PICkit3 programming. Return only bits 5:0 for pointer 0x00. Add pointer 0x04 as a one byte device status register. Include I²C related status or fault information, such as MSSP receive overflow or write collision. Choose your own status encoding and handling; document what each values means and how it is cleared. You many also report other useful conditions, such as ADC conversion busy/ready state, invalid data, or application faults. Read the status register after normal transactions and explain the results.

11/12/2026
LRG

Lab 10 - I²C

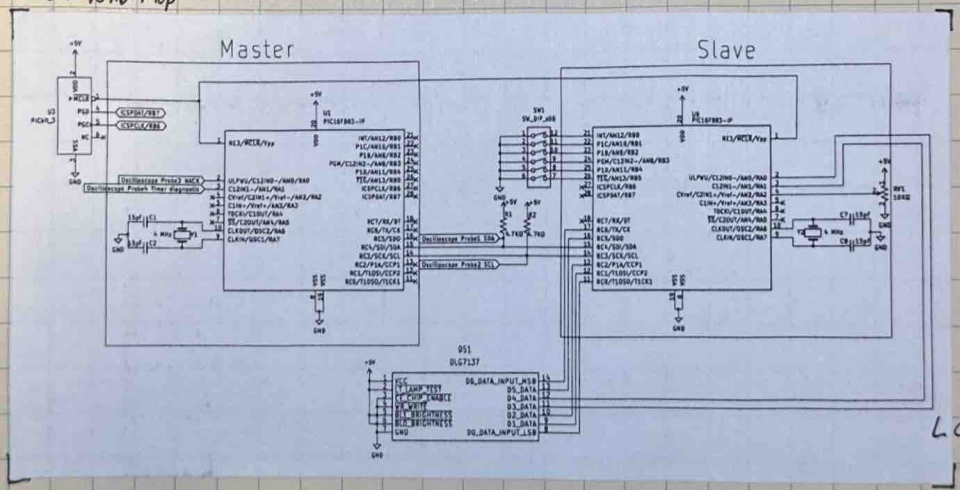
```
Select PORTB: START → 0x40 → ACK → 0x00 → ACK → STOP
Read PORTB:   START → 0x41 → ACK → switch byte → PIC NACK → STOP

Select status: START → 0x40 → ACK → 0x04 → ACK → STOP
Read status:   START → 0x41 → ACK → status byte → PIC NACK → STOP
```

LG

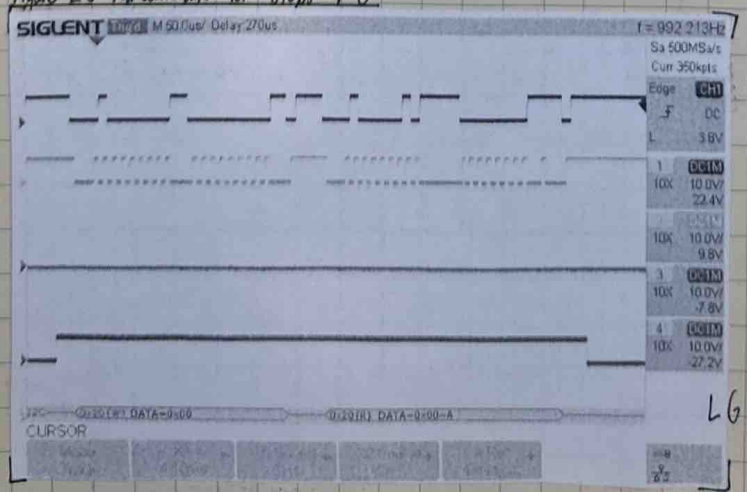
-Change several DIP-switch patterns. Compare the actual switch positions, returned six-bit value, and oscilloscope decode. Keep the status read working as you add later features. Clock-stretching experiment. After the PORTB read works normally, add a short time inline delay in the slaves PORTB read path, while its MSSP hardware is holding SCL LOW and before it releases SCL to send the data byte. Compare a normal read with several progressively longer delays, for example 0, 100µs, 1ms, and 10ms. Calculate or measure the actual delay; do not assume that a software delay produces clock stretching unless the scope shows SCL held LOW. Capture SDA and SCL and measure the extended SCL Low time for each delay. Does the master wait and receive the correct PORTB byte? If the master has a timeout, determine approximately when it reports a failure and compare that limit with the measured stretch. If it has no timeout, explain how a stalled read behaves and how you regain control. Record the master's result for each test. Restore normal PORTB operation before proceeding

0 → 0 → Yep
100µs → 100µs → Yep
1ms → 998µs → Yep
10ms → 10ms → Yep



LG

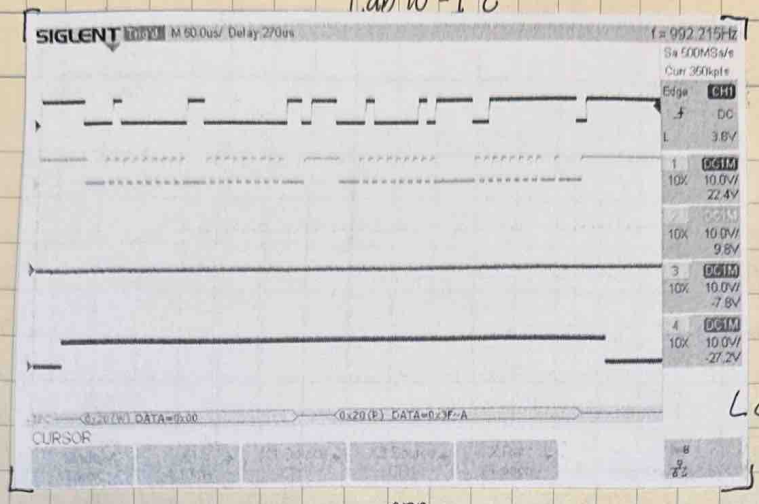
Figure 28: Full schematic for steps 4-6



LG

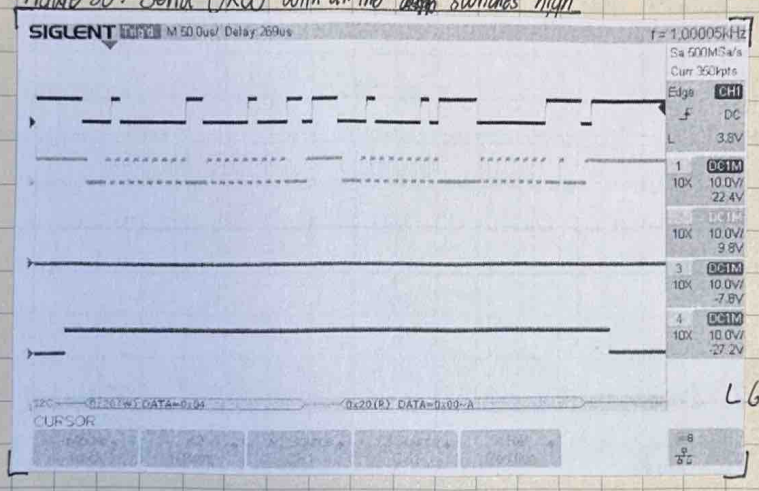
Figure 29: Send 0x00 with all dip switches to GND

11/3/2026
LRG



LG

·Figure 30: Send 0x00 with all the DIP switches high



LG

·Figure 31: Pointer 0x04 to get a status byte 0x00

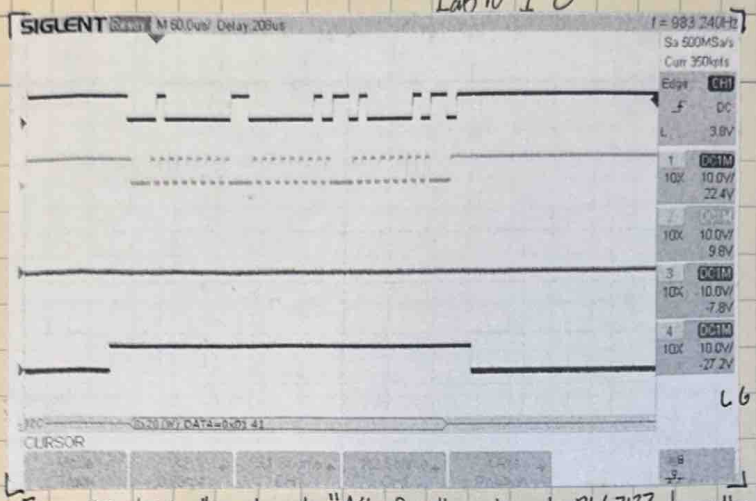
5) Write a character to the DLG7137. Use the DLG7137 single character display from Lab 02. it accepts seven bit ASCII; the slave must drive its data lines and pulse its active-low WR input. RC3 and RC4 remain dedicated to I²C, so don't reuse the original PORTC wiring unchanged. Modify the DLG7137 wiring to use two available PORTA pins. Document and explain the change. Check the DLG7137 data sheet for orientation, control pins, power, and write timing. Document the wiring and decoupling. A write to pointer 0x01 followed by ASCII 0x41 should display A:

```
START → 0x40 → ACK → 0x01 → ACK → 0x41 → ACK → STOP
```

LG

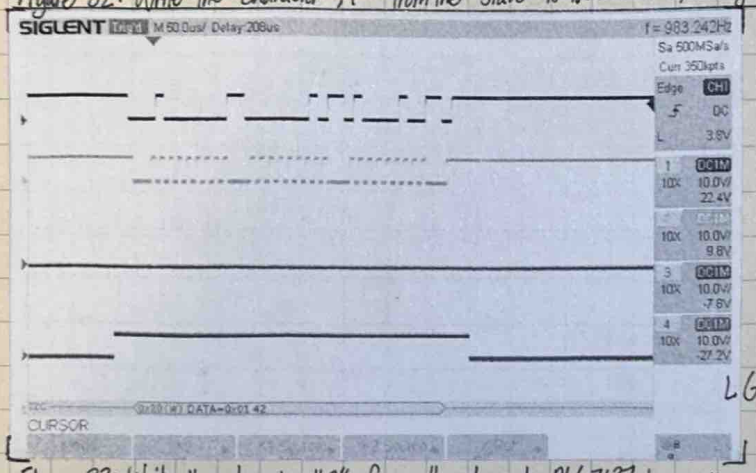
-Stage the received character and update the display outside the time-sensitive I²C event handler. Repeat with a different printable ASCII character. Confirm the PORTB and status read still work.

LG



LG

- Figure 32: Write the character "A" from the slave to DLG7137 by sending 0x41 from master



LG

- Figure 33: Write the character "B" from the slave to DLG7137 by sending 0x42 from master

6) Read the slave's ADC. Connect a potentiometer between the slave's VDD and GND, with the wiper on RA2/AN2 (pin 4). Configure the PIC's 10-bit ADC, VDD/VSS references, and right-justified result. Use a timer to sample periodically without waiting in a long blocking loop. Polster 0x02 returns a consistent two byte sample: upper two ADC bits in the first byte, lower eight bits in the second. Store conversions in RAM and ensure that I²C read bytes cannot come from different ADC conversions.

G

```
Select ADC: START → 0x40 → ACK → 0x02 → ACK → STOP
Read ADC:   START → 0x41 → ACK → ADC high → PIC ACK → ADC low → PIC NACK → STOP
```

- Combine the received bytes and estimate the input voltage using the measured supply voltage:

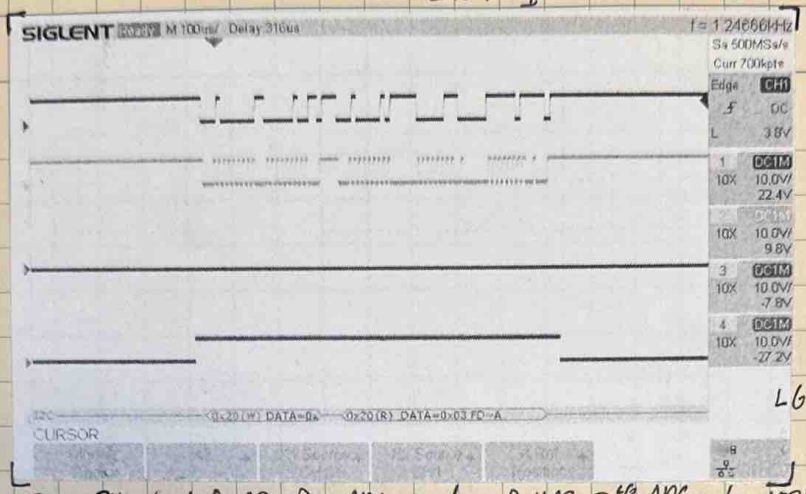
$$ADC_{10} = (C_{high} \,\&\, 0x03) << 8) \mid low$$

$$V_{AN2} = \frac{ADC_{10}}{1024} \cdot V_{DD}$$

- Measure the actual AN2 voltage at GND, approximately midpoint, and VDD settings. Compare each measurement with the I²C result (approximately 0, 512, and 1023 counts, respectively). Allow time for a valid conversion after reset. Verify that the scope shows the master's ACK between data bytes and NACK after the last byte.

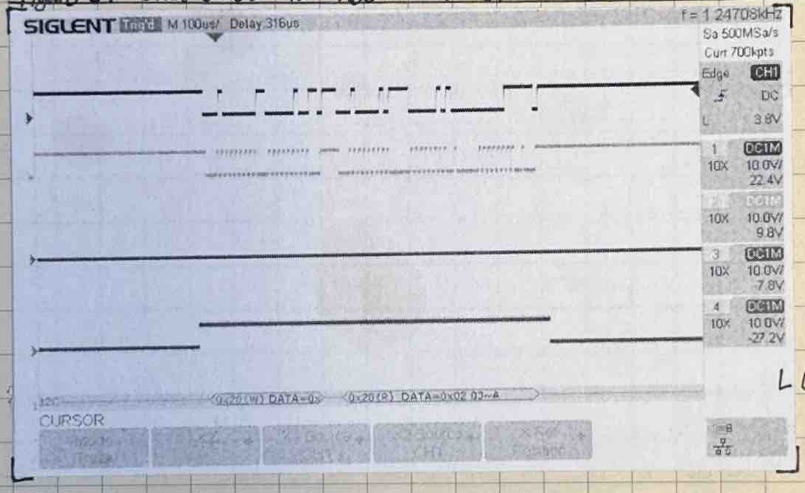
- 0 → 0v → 0v → 0
- 512 → 2.5v → 2.501v → 517
- 1023 → 5v → 4.93v → 1021

expected measured



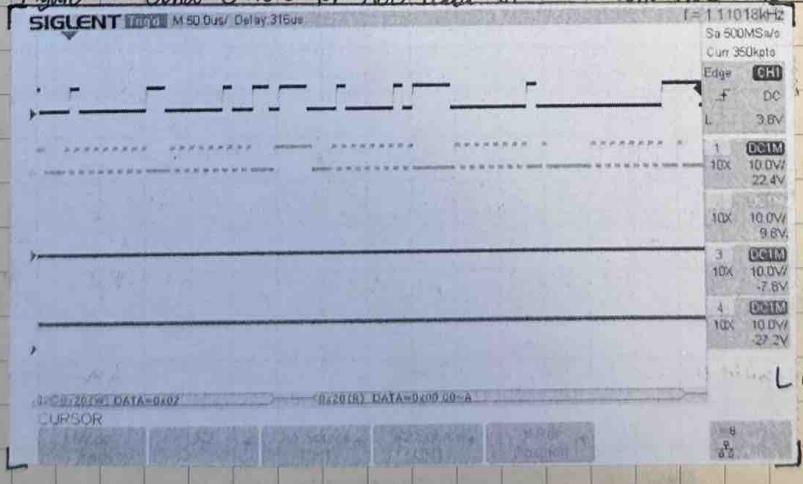
LG

•Figure 34: Send 0x02 for ADC read of 4.98v @ LG ADC value 1021



LG

•Figure 35: Send 0x02 for ADC read of 2.509 volts ADC value of 513



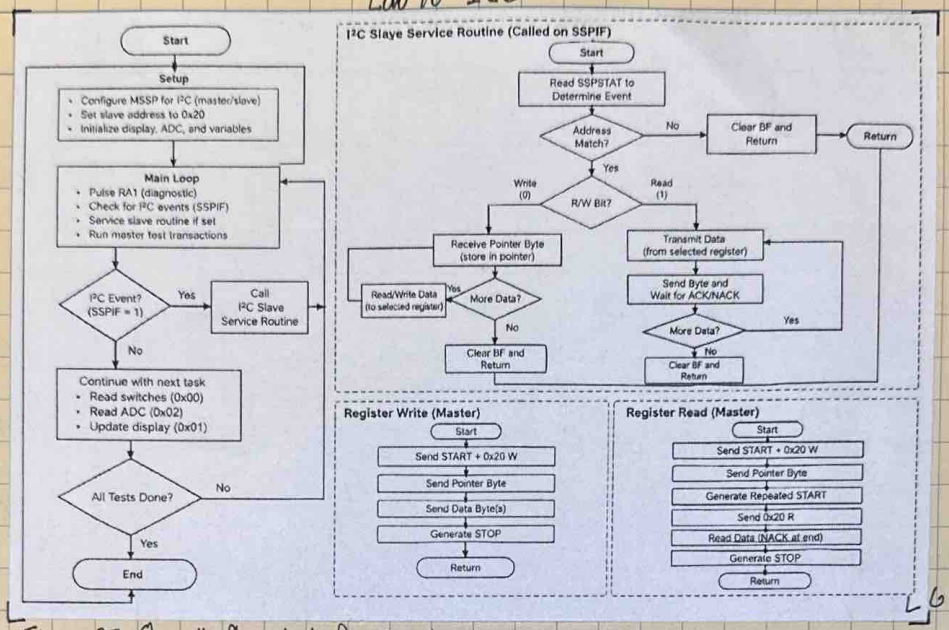
LG

•Figure 36: Send 0x02 for ADC read of 0v ADC value of 0

•Link to master and slave codes are below due to the amount of code see the links for full code with GPR's, SFR's, and full code:

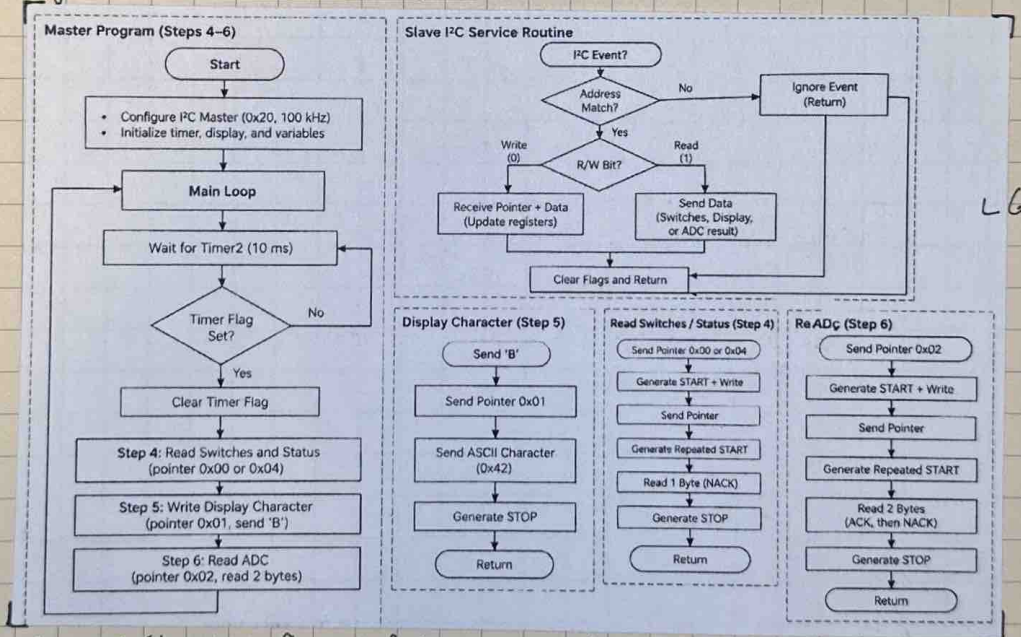
-https://github.com/groelwke/Lab_10_I2C/tree/main/LAB_10_Part_3_I2C.X (this is the master)

-https://github.com/groelwke/Lab_10_I2C/tree/main/LAB_10_Part_3_I2C_Slave.X (this is the slave)



LG

•Figure 37: Overall flow chart for all code



LG

•Figure 38: Steps 4-6 flow chart fo ADC, send character, and dip switch control.

TJR
11-4-26

•Part 4-UART-to-I2C Bridge:

•Goal: Turn the master PIC16F883 into a generic PC/UART-to-I²C bridge controlled by a GUI. The GUI sends custom byte sequences, read responses, discovers I²C devices, and records communication. Test it with the Part 3 PIC slave and Part 2 ADS1115. Use the 4 MHz master, 100 kHz I²C, and 9600 8N1 UART with a MAX232 or equivalent RS-232/TTL interface. Support one host request at a time and ordinary 7-bit addresses. Separate STOP-terminated writes and reads and sufficient. Repeated START, general call, 10-bit address, and multi-master operation are not required. The packet tables below specify the interface you must implement in Part 4. Choose your own routines, buffers, and error-handling structure while producing the specified externally visible behavior.

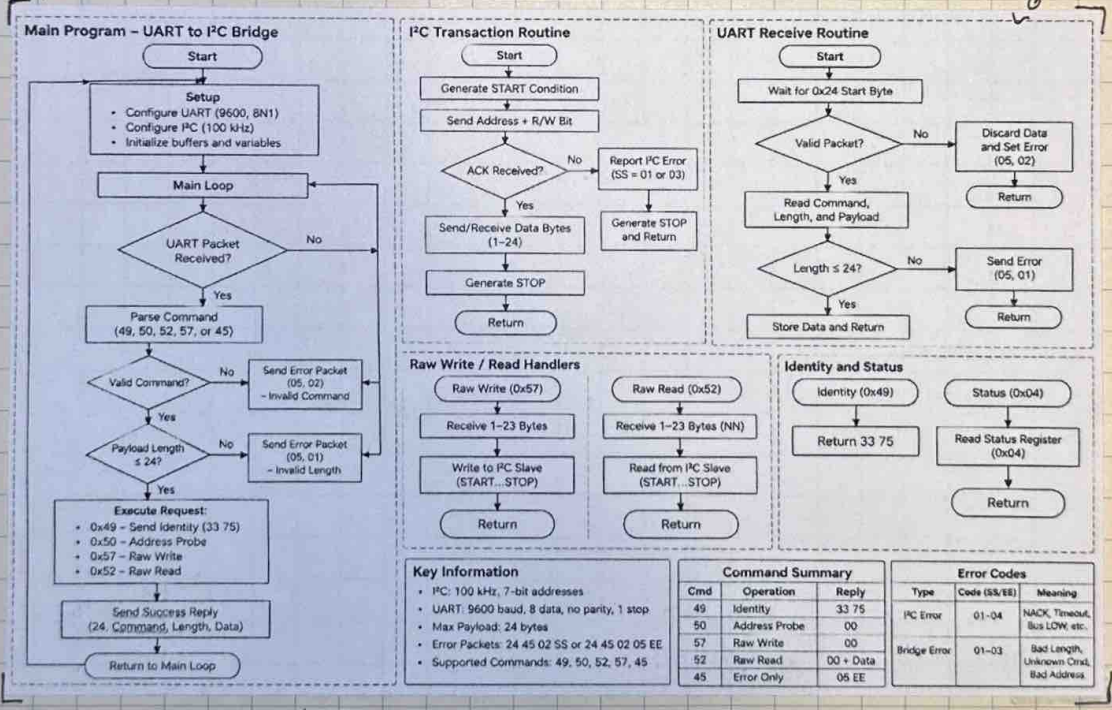
11/4/2026
LRG

Lab 10 - I²C

Before Lab:

- An updated schematic showing the PC serial adapter, MAX232, master PIC, both existing I²C devices, common ground, power, pull-ups, and compatible logic levels. Account for pull-ups on the ADS1115 breakout.
- Flowcharts for receiving a UART frame, validating the request, performing an I²C transaction, and returning the result. Document the EUSART and MSSP registers you use.
- A command/response packet table and test cases for ACK, NACK, malformed requests, and transfer-length limits.
- A GUI plan covering hex/text entry, saved transmissions, live UART/I²C logs, a 128-address scan, and automatic bridge identification/reconnection.

-Reuse the Part 3 master routines and UART framing from Lab 8. Keep UART transport, I²C transactions, and bridge application behavior seperate. Any clear PIC-AS source-file organization is acceptable.



- Figure 39: PART4 UART-to-I²C flow chart

- **UART packet format and error packets:** UART packets contain binary bytes, shown here in hexadecimal. Each frame is 24, one byte command byte, a one-byte payload length, then that many payload bytes. Implement a maximum accepted payload of 24 bytes. The bridge must reject invalid lengths without overflowing its buffer or initiating I²C traffic.

Command byte	Operation	Host payload	Successful reply payload
49	Identity	None	33 75
50	Address probe	AA	00
57	Raw write	AA DD...	00
52	Raw read	AA NN	00 DD...
45	Error response only	None	Two error bytes (below)

- Table 8: Command byte table with operation

11/4/2026
LRG

-RAW writes contain 1-23 data bytes; raw reads request 1-23 bytes. Successful replies include the full 24, command, length, and payload. On failure, return one error packet instead of a successful command reply. The bridge sends the host's write bytes unchanged and does not interpret I²C devices registers. A pointer write and subsequent read are seperate requests. Preserve the Lab 8 UART Framing and overrun error packets. Add seperate I²C and bridge-validation error categories without reusing those codes. Every error packet has five bytes: 24 45 02, error category detail.

Complete error packet	Meaning
`24 45 02 01 00`	UART framing error, retained from Lab 08
`24 45 02 02 00`	UART receive overrun, retained from Lab 08
`24 45 02 04 SS`	New I²C error; `SS` identifies the cause
`24 45 02 05 EE`	New bridge request/validation error; `EE` identifies the cause

LG

•Table 9: Complete error packet for UART + I²C

I²C error detail (SS)	Meaning	Bridge error detail (EE)	Meaning
`01`	Address/data NACK	`01`	Invalid length or count
`02`	I²C timeout	`02`	Unknown command
`03`	Bus LOW / not idle	`03`	Invalid 7-bit address
`04`	SCL held LOW		

LG

•Table 10: I²C error detail for what each error means

-Preserve Lab 8's UART receiver recovery behavior: discard a damaged partial packet, clear the ~~receive~~ receive fault, and accept the next valid request. If an I²C operation fails, stop/clean up the bus when possible, report the error and allow a later request. A request produces one response: either its normal success packet or a five-byte error packet.

LG

Symbol	Meaning
`AA`	Seven-bit I²C device address
`NN`	Number of bytes requested by a read
`SS`	I²C error detail
`EE`	Bridge error detail
`VV`	PORTB switch byte
`HH`	High result byte
`LL`	Low result byte
`DD`	One variable data byte
`× N`	Repeat the preceding byte/sequence N times (not transmitted)
START / STOP	I²C bus conditions
ACK / NACK	I²C ninth-bit responses, not data bytes

•Table 11: Legend for all transaction tables

- All other two-digit bytes LG values are literal hexadecimal bytes. The Part 3 PIC slave retains its register pointer across STOP, so a pointer write can be followed by a seperate read.

- Expected UART TX/RX and I²C transactions LG - UART TX: host → bridge. UART RX: bridge → host. The Part 3 PIC slave has assigned 7-bit address 20, so its I²C address bytes are 40 for write and 41 for read. The examples using 21 as an absent address assume no device responds at 21 on your bus; choose another unused ordinary address if necessary.

Test	UART TX	UART RX	Expected I²C transaction
Identify	24 49 00	24 49 02 33 75	None
Probe PIC, ACK	24 50 01 20	24 50 01 00	START, 40, ACK, STOP
Probe absent 21	24 50 01 21	24 45 02 04 01	START, 42, NACK, STOP
Display character 41	24 57 03 20 01 41	24 57 01 00	START, 40, ACK, 01, ACK, 41, ACK, STOP
Select PORTB	24 57 02 20 00	24 57 01 00	START, 40, ACK, 00, ACK, STOP
Read PORTB	24 52 02 20 01	24 52 02 00 VV	START, 41, ACK, VV, NACK, STOP
Select PIC ADC	24 57 02 20 02	24 57 01 00	START, 40, ACK, 02, ACK, STOP
Read PIC ADC	24 52 02 20 02	24 52 03 00 HH LL	START, 41, ACK, HH, ACK, LL, NACK, STOP

LG

- Table 12: Tests for UART RX/TX transactions with I²C

- Each pointer write and subsequent read is a separate UART requesent and a separate I²C transaction, with STOP between them. For the ADS1115 examples, 48 is a sample 7-bit address, with I²C address bytes 90 (write) and 91 (read). Substitute your verified Part 2 address and its corresponding address bytes.

Test	UART TX	UART RX	Expected I²C transaction
Probe ADS1115	24 50 01 48	24 50 01 00	START, 90, ACK, STOP
Select Config	24 57 02 48 01	24 57 01 00	START, 90, ACK, 01, ACK, STOP
Read Config	24 52 02 48 02	24 52 03 00 HH LL	START, 91, ACK, HH, ACK, LL, NACK, STOP
Select Conversion	24 57 02 48 00	24 57 01 00	START, 90, ACK, 00, ACK, STOP
Read Conversion	24 52 02 48 02	24 52 03 00 HH LL	START, 91, ACK, HH, ACK, LL, NACK, STOP

LG

- Table 13: Tests continued

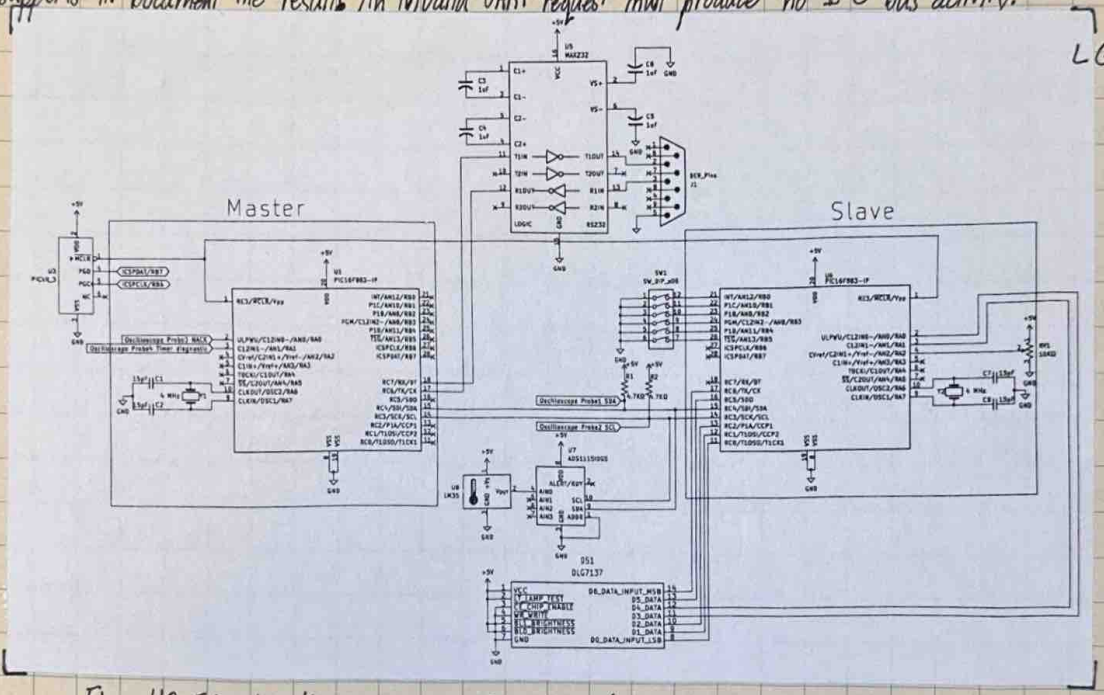
Test	UART TX	UART RX	Expected I²C transaction
Write NACK, absent 21	24 57 02 21 00	24 45 02 04 01	START, 42, NACK, STOP
Read NACK, absent 21	24 52 02 21 01	24 45 02 04 01	START, 43, NACK, STOP
Bad probe length	24 50 00	24 45 02 05 01	None
Unknown command	24 51 00	24 45 02 05 02	None
Invalid address	24 50 01 00	24 45 02 05 03	None
Zero-byte read	24 52 02 20 00	24 45 02 05 01	None
Address-only write	24 57 01 20	24 45 02 05 01	None
Invalid 24-byte read	24 52 02 20 18	24 45 02 05 01	None
Write with 24 data bytes (oversized)	24 57 19 20 01 + 41 × 23	24 45 02 05 01	None
23-byte read (device supports it)	24 52 02 20 17	24 52 18 00 + DD × 23	START, 41, ACK, (DD, ACK) × 22, DD, NACK, STOP
23-byte write (device supports it)	24 57 18 20 01 + 41 × 21 + 42	24 57 01 00	START, 40, ACK, 01, ACK, (41, ACK) × 21, 42, ACK, STOP

LG

- Table 14: Errors and transfer boundaries

- Upper-limit examples are conditional: the last two rows show what a successful 23-byte transaction would look like if the selected slave supports that many consecutive bytes. Part 3 doesn't require such streaming behavior. You do not have to modify the slave solely for these tests. Verify the bridge's length validation at the boundary; demonstrate a complete maximum-length transfer if a connected slave supports it. Document the results. An invalid UART request must produce no I²C bus activity.

LG



- Figure 40: Full schematic for UART-to-I²C communication

- In the LAB:

1) Start the host GUI and UART link. Extend lab 8 host application or create a new GUI with serial-port communication and raw hex transmission. Add UART to the working master project. From your GUI, send 24 49 00 and verify 24 49 02 33 75. Handle responses that arrive in multiple serial reads.

2) Address probes. Implement probe command 60. Probe the PIC slave at 20 and an address you have confirmed is unoccupied. Confirm ACK/NACK with decoded START, address, and STOP on the scope.

3) Generic write/read. Implement write command 57 and read command 52 using existing I²C transaction routines and bounded buffers. Send 24 57 03 20 01 41 to display A. Write pointer 0x00, read PORTB and change a switch. Write pointer 0x02, read the two-byte PIC ADC and compare with potentiometer voltage.

4) Second I²C device. Connect the ADS1115 and PIC slave at the same time. Probe each at its verified address. From the host, select and read ADS1115 Config and Conversion registers using separate write (57) and read (52) calls, reusing Part 2 setup if needed. Change the input voltage. Confirm ability to switch between slaves.

5) Limits and errors. Use the GUI to try zero-length, one byte, ordinary, maximum-length and oversized requests, including a zero byte read, an address-only write, a 23-byte read request (17), a 24-byte read request (18) and a write with 24 data bytes (25-byte payload, above the bridge limit). Record the raw request, reply or timeout, whether the bridge rejected it, and any I²C activity. Test NACK on an unoccupied address and restore operation after a UART framing or overrun error as in Lab 8. Out-of-range requests must not cause buffer overflow or I²C activity. Complete a 23-byte transfer only if your slave supports it; don't extend the Part 3 slave for this test.

11/5/2026
LRG

Lab 10-I²C

6) Complete the host GUI. Adding the remaining features below. Use your GUI to repeat the PIC and ADS1115 transactions, scan all addresses, save, and replay commands, then test automatic connection and recovery.

•Host GUI requirements: Build a graphical application, extending Lab 8 or using your own design:

•Custom transmission and lengths: Select an I²C address, probe, read count, or write data. Enter write data as hex bytes (for example, 01 41) or text encoded as UTF-8; display the actual bytes and length before sending. Both modes supply the data portion of a framed write, not unframed UART text. Allow users to deliberately test zero, short, maximum and oversized payloads lengths and read counts, including values outside the bridge's accepted range. Show a warning but do not truncate, split, or prevent a deliberate boundary test. Provide an advanced raw-frame mode for testing declared length mismatches and unsupported commands. Reject input that cannot be represented as bytes

•Saved transmissions: Name, save, select, edit, delete, and retransmit command sequences. Preserve them across application restarts, including address, operation, payload/read count, and hex/text mode.

•Time-stamped history and file logging: Display and append every UART transmitted & transmission and every received reply/error frame to a log file, including automatic identity checks and address scans. Include timestape (date and time), direction, port, raw hex bytes, payload length, and decode response when available. Also log all errors detected by the GUI or reported by the bridge, including Lab 8 framing/overrun packets, new I²C error packets, invalid requests, serial exceptions, malformed/incomplete replies and response timeouts. Timestamp disconnection/reconnection events and indicate which request failed.

•I²C address discovery: Provide a Scan button that examines all 128 seven bits address values, 00–7F, and shows progress and a list of acknowledged slave addresses in hexadecimal. The ranges 00-07 and 78-7F are reserved for special I²C purposes. Mark those values Reserved without sending an I²C transaction. For each ordinary address 08–77, send a probe command (50) and classify the result as ACK, NACK, or communication error. Do not treat a skipped reserved address as a NACK.

•Automatic connection: At setup, search available serial port at 9600 8N1 and accept the bridge only after the exact identify reply 24 49 02 33 75 to 24 49 00. Check bridge connectivity approximately once per second after communication loss. Reconnect automatically, including if the port name changes. Never send device commands to an unidentified port.

•Serial coordination: Allow only one outstanding command at a time, including address scans and connectivity checks. Reassemble fragmented UART replies, enforce a response timeout and keep the GUI responsive. Queue or defer the one-second identify check while another request is active.

•Test order:

1) 24 49 00 should return 24 49 02 33 75.

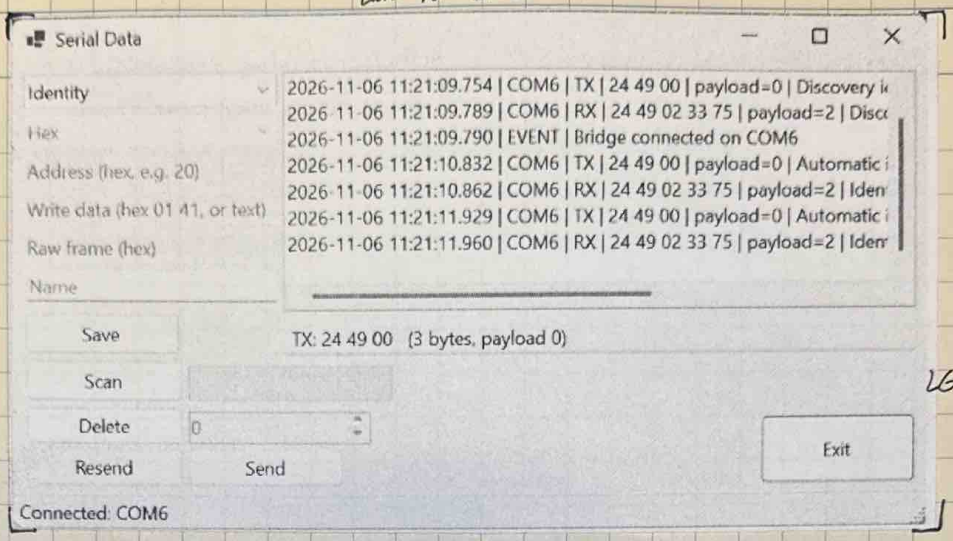
2) 24 50 01 20 should ACK, and 24 50 01 21 should return 24 45 02 04 01.

3) 24 57 03 20 01 41 shows 'A'. The 24 57 02 20 00 followed by 24 52 02 20 01 reads PORTB.

4) Run bad length and bad address cases from the handout table.

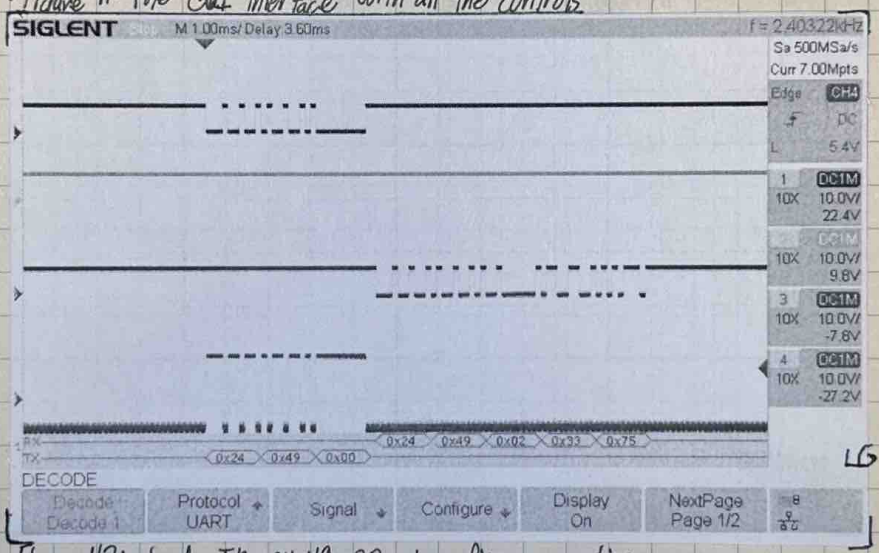
5) For error 04, briefly ground SCL through a jumper

11/5/2026
LRG



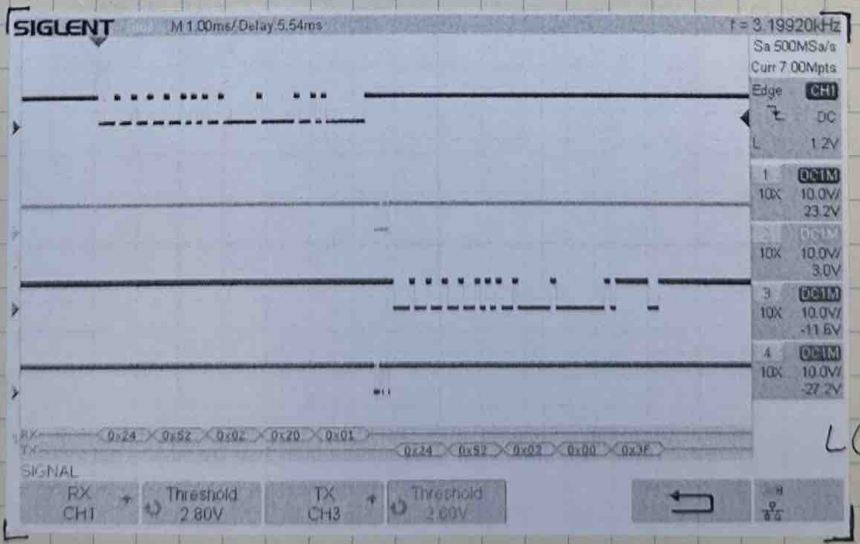
LG

• Figure 41: The GUI interface with all the controls



LG

•Figure 42: Send ID 24 49 00 to confirm connection

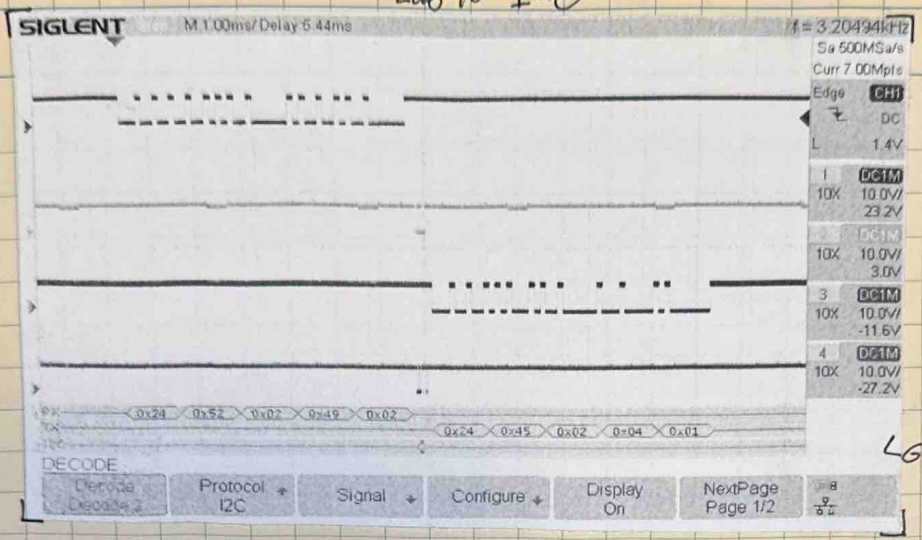


LG

•Figure 43: Request ALL from Slave

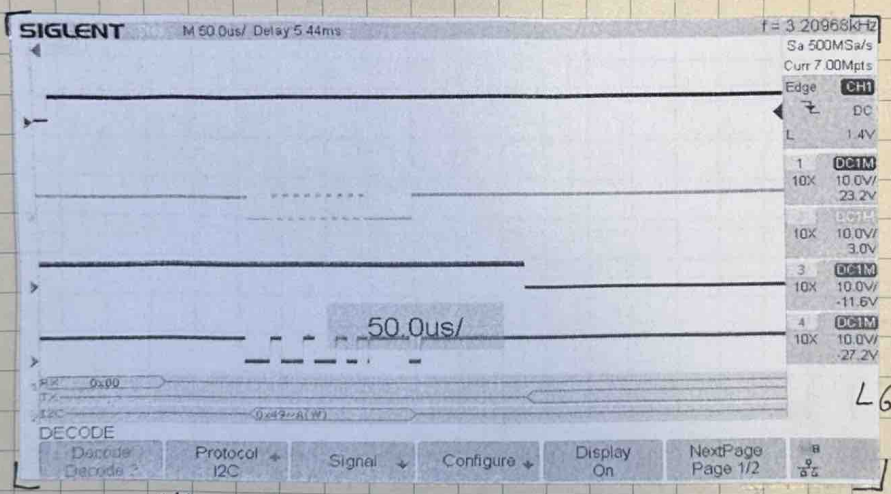
Lab 10 - I2C

11/6/2026
LRG



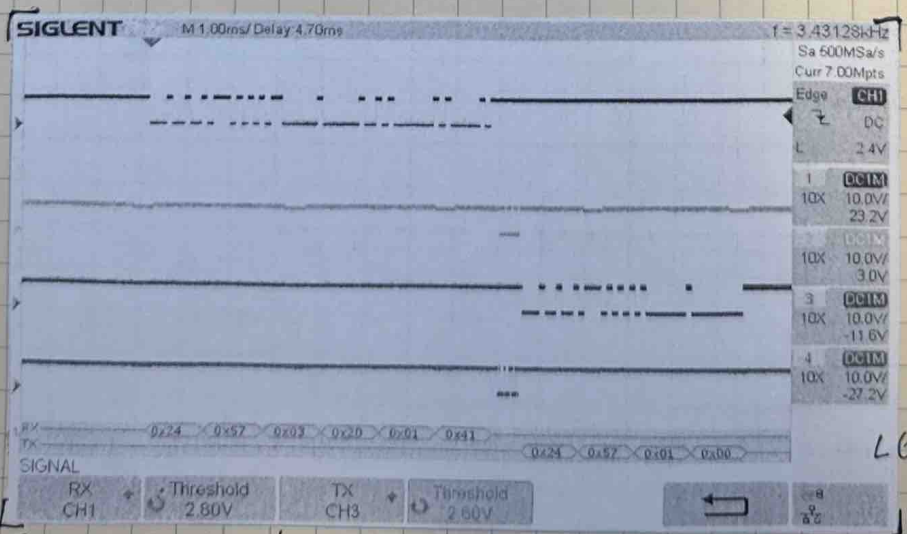
LG

- Figure 44: Send wrong information for error



LG

- Figure 45: NACK Shown when there is no slave at 0x49 address



LG

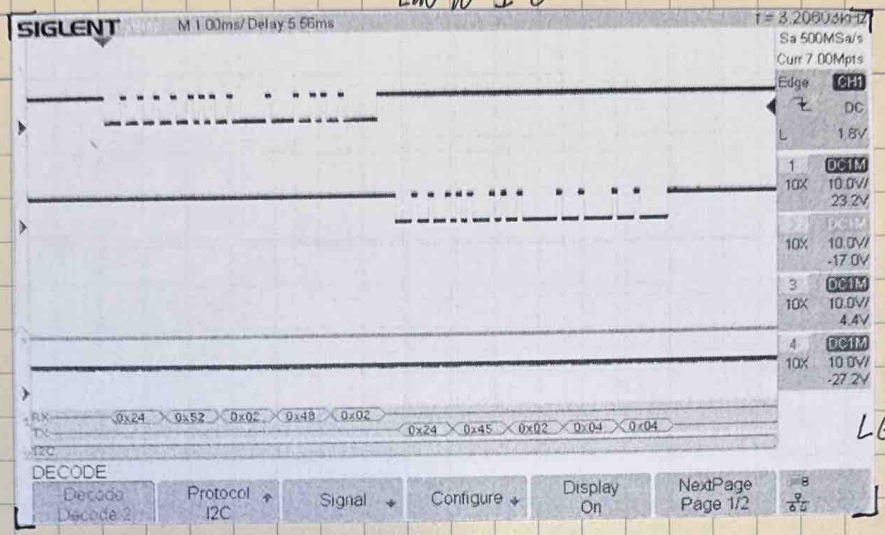
- Figure 46: Send value to display A

- Link to code & GUI -

- https://github.com/groeluke/Lab_10_I2C/tree/main/LAB_10_Part_4_I2C.x
- https://github.com/groeluke/Lab_10_I2C/tree/main/SerialData

TSR
11-6-26

11/6/2026
LRG



LG

Figure 47: Capture of I²C SCL hooked to ground error

LG

Conclusion:

In this Lab we were asked to hook up a master and slave via using the PIC16F883 (MSSP) Master Synchronous Serial Port peripheral for I²C communication. To start this lab Part 1 to create the setup for an I²C Master and Bus Timing to create a master to send a periodic address-only write transaction with no slave connected. In this part the program was verifying that SDA and SCL communication was happening. Checking that the proper pull-up resister were installed. Showing a Start transaction, address 0x20, R/W=0, ninth-bit NACK, and the STOP of transmission. Part 2 was proving communication with a slave board ADS1115 verifying its different addresses and there configuration. In this Part there was 6 steps to verify many different aspects of the ADS 1115 and what it can do. Part 3 was creating a slave and master communication using the PIC16F883, by running each code via 2 different pic's. By developing a slave at 7-bit address 0x20 and extending Part 2's master code to test the slave. The requirements were to make the slave code so when the master step is selected the slave will return either, reading its digital inputs, write to a display via a DLG7137 characters, and retriever 10-bit ADC measurements. There was also 6 steps to verify and test. Part 4 Was not only combing Part 2 and 3's slaves to cor LG create and LG an actual communication interface. By turning the master PIC16F883 into a generic PC/UART-to-I²C bridge controlled by a windows form app GUI interface. By integration of 2 slaves and creating a scanning to check what slaves and addresses are available. Also including 6 steps testing each of the addresses and comands to verify connection with them and controls. To write many different characters and being able to verify different transactions. I²C uses the SDA and SCL pins (PIN 14 = SCL & PIN 15 = SDA) with pull-up resistors. Devices only pull the lines low, never driving them high, many devices can share the bus without fighting each other. The master generates the clock and starts every transaction with a START and the 7-bit address plus a R/W bit. The receiver pulls SDA low on the ninth clock to ACK each byte, and a NACK means no device answered or the transfer is over.

LG